



Technische Hochschule Mittelhessen

und

Spacelfight Rocketry Giessen e.V.

Projektarbeit

Development of the Electronic Actuaion & Guidance for Landing and Elevation (EAGLE) system

---

Entwicklung des Electronic Acutation & Guidance for Landing and Elevation (EAGLE) Systems

SoSe 2025

vorgelegt von

Jason Gläser

betreut durch

Prof. Dr. Chris Volkmar

October 29, 2026

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig und unter aus schließlicher Verwendung der angegebenen Literatur und Hilfsmittel erstellt habe. Ergänzend erkläre ich, dass bei der Erstellung dieser Arbeit generative Werkzeuge der Künstlichen Intelligenz (KI) als Hilfsmittel zum Einsatz kamen. Der genaue Umfang, der Verwendungszweck sowie die betroffenen Kapitel und Tools sind im beigefügten „KI-Verzeichnis“ (Appendix 12.1) vollständig und wahrheitsgemäß offengelegt.Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und nicht veröffentlicht.

Ort, Datum: ..... Unterschrift: .....

Abstract

The development of the "Electronic Actuation & Guidance for Landing and Elevation" (EAGLE) system is a project developed in cooperation with the student rocketry association "Spaceflight Rocketry Gießen e.V." as part of its self-developed on-board computer called "Advances System for Control Electronics, Navigation and Tracking" (ASCENT). Therefore, I would like to thank Spaceflight Rocketry Giessen for giving me the opportunity to pursue the project with their equipment and funding.
EAGLE is a avionics module which uses real-time sensor data to accurately estimate the rockets position and attitude as well as detecting mission critical waypoints (such as apogee) to deploy the recovery system, perform the staging and subsequent firing of the sustainer's motor and make real-time decisions about an active aerodynamical airbrake system, which accurately limits the maximum altitude to a preset value.
The development process of a printed circuit board for this system according to the rule book provided by the "European Rocketry Challenge" will be the core subject of this project.

Contents

List of Figures V

List of Tables VII

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 ARCHER . . . 2
- 1.3 Objectives and Scope . . . 2

2 Theoretical Basics 3
- 2.1 Electrical Design . . . 3
  - 2.1.1 RC-Based Voltage Stabilization of Integrated Circuits . . . 3
  - 2.1.2 Metal-Oxide-Semiconductor Field-Fffect Transistor (MOSFET) . . . 5
  - 2.1.3 Inverting Schmitt Trigger . . . 5
- 2.2 Software Concepts . . . 6
  - 2.2.1 State Machines . . . 6
  - 2.2.2 PID Controllers . . . 7
- 2.3 PCB Design . . . 7
- 2.4 Communication Protocols . . . 7
  - 2.4.1 Inter-Integrated Circuit (I^2C) . . . 7
  - 2.4.2 Universal Asynchronous Receiver-Transmitter (UART) . . . 8
  - 2.4.3 Serial Peripheral Interface (SPI) . . . 9
  - 2.4.4 Comparison . . . 10
- 2.5 Sounding Rocket Recovery Systems . . . 10
- 2.6 Flight Trajectory . . . 12
- 2.7 Barometric Formula . . . 13
  - 2.7.1 Hydrostatic equation . . . 13
  - 2.7.2 Isothermal barometric formula . . . 14
  - 2.7.3 Adiabatic barometric formula . . . 15
  - 2.7.4 International barometric formula . . . 16
- 2.8 Nyquist Criteria . . . 16
- 2.9 Avionics Architecture for Recovery Systems . . . 16

3 Background and Related Work 17
- 3.1 DLR - MORABA . . . 17
- 3.2 European Student Rocketry . . . 18
  - 3.2.1 Bundesverband studentischer Raumfahrt (BVSR) . . . 19
  - 3.2.2 European Rocketry Challenge . . . 19

4 Requirement Analysis **21**
4.1 Control Output and Power Requirements . . . 21
4.2 Sensor Requirements . . . 21
4.3 EuRoC system requirements . . . 22
4.3.1 Ignition Systems . . . 22
4.3.2 Recovery Systems and Avionics . . . 23
4.3.3 Active flight control systems . . . 23

5 System Architecture **24**
5.1 Hardware Architecture . . . 24
5.1.1 ASCENT II Architecture . . . 24
5.1.2 PCB Concept . . . 26
5.1.3 EAGLE Architecture . . . 27
5.2 Software Architecture . . . 28
5.2.1 State Machine . . . 28
5.2.2 State Transitions . . . 30
5.2.3 PID Controllers . . . 31
5.2.4 High-Level Code Architecture . . . 32
5.2.5 Main Loop Architecture . . . 32

6 Preliminary Design Phase **34**
6.1 Component Selection . . . 34
6.2 Microcontroller . . . 35
6.2.1 AVR128DB64 . . . 35
6.2.2 ESP32-S3-Vroom-N4-1u . . . 35
6.3 Design Evaluation and Revision . . . 35

7 Software Implementation **36**
7.1 Internal Data Processing . . . 36
7.1.1 Integration of IMU data . . . 36
7.2 State machine . . . 36

8 PCB Design - ASCENT II **38**
8.1 Power Budget . . . 38

9 Verification and Testing **39**

10 Discussion **40**

11 References **41**

12 Appendix **II**
12.1 Artificial Intelligence . . . II
12.2 Tables . . . III
12.2.1 State transition truth tables . . . III
12.2.2 Sensor characteristics . . . V

List of Figures

2.1.1 figure 4
2.1.2 Typical functionality of inverting Schmitt-Triggers[5] 5
2.4.1 figure 9
2.5.1 figure 12
2.7.1 figure 14

3.1.1 figure 18

5.1.1 ASCENT II architecture 24
5.1.2 PCB concept 26
5.1.3 EAGLE architecture 28
5.2.1 Software architecture 29
5.2.2 Software architecture 33

List of Tables

2.4.1 Comparison of communication protocols 10

12.1.1AI usage within this thesis II
12.2.1EAGLE setup state transitions III
12.2.2EAGLE flight state transitions IV
12.2.3Inertial Measurement Unit(IMU) characteristics Nr.1 V
12.2.4Inertial Measurement Unit(IMU) characteristics Nr.2 VI
12.2.5Accelerometer characteristics VII
12.2.6Pressure sensor characteristics VIII

List of Abbreviations

SPROG ..... Spaceflight Rocketry Giessen

EAGLE ..... Electronic Acutation & Guidance for Landing and Eleveation

PIPE ..... Project for Inital Practical Experimentation

ARCHER ..... Aerial Rocket for Competions and High-altitude Experimental Rocketry

PCB ..... Printed Circuit Board

PWM ..... Power Width Modulation

IC ..... Integrated circuit

PSU ..... Power supply unit

SMD ..... Surface mounted devices

SPI ..... Serial peripheral interface

I2C ..... Inter-Integrated Circuit

UART ..... Universal Asynchronous Receiver-Transmitter

RAM ..... Random access memory

UPDI ..... Unified Program and Debug Interface

GND ..... Ground

ASCENT ..... Advanced System for Control Electronics, Navigation and Tracking

EUROC ..... European Rocketry Challenge

COTS ..... Commercial Off-The-Shelf

JLU ..... Justus-Liebig-University Giessen

THM ..... Technische Hochschule Mittelhessen

OSR ..... Oversampling ratio

MOSFET ..... Metal-oxide-semiconductor field-effect transistor

ST ..... Schmitt Trigger

uC ..... Microcontroller

PSS ..... Pressure Sensor

IMU ..... Inertial Measurement Unit

1 Introduction

"Spaceflight Rocketry Gießen e.V." is a student-founded and ran rocketry association at the Justus Liebig University and University of Applied Sciences in Giessen, which develops and builds experimental sounding rockets, with which the association wants to compete at international student rocketry competitions such as the "European Rocketry Challenge" (EuRoC) in Portugal.

1.1 Motivation

Most aerospace related study programs consist of mostly mechanical engineering courses. This leads to most german student teams to have a lack of electrical understanding resulting in the use of commercial-of-the-shelf (COTS) avionics modules such as the CATS-Vega, an all-in-one board computer. Due to the study program "Physics and Technology for Space Applications" at Justus Liebig University in Giessen, which heavily focuses on electrical engineering in the context of space exploration, SPROG's avionics team has been its largest team from the beginning. It was therefore decided to develop its own avionics module called "Advanced System for Control Electronics, Navigation and Tracking" (ASCENT), which consisted of modularized PCBs with tasks ranging from telemetry to recovery.
After roughly a year of development, SPROG launched its first ever rocket "PIPE" in Karlshöfen near Bremen in August 2024. During the launch campaign, the self-developed avionics, specifically the recovery module, showed major issues which were not observed during testing. This was overlooked mostly due to limited testing ahead of the launch campaign because of time restrictions and lack of funding.
Especially in the early phases of a rocketry project, it is important to collect as much data as possible. To ensure this, it is imperative to have a working recovery system to allow for the recovery of the onboard data drives for data analysis. Whilst the telemetry can provide a selected amount of data, the full data set can only be retrieved by actually getting the data drives from the vehicle. To ensure that they do not corrupt or crack, the recovery system has to reliably deploy and slow down the vehicle enough so that the rocket does not break upon landing.
In aerospace, reliability is more important than ever, because the launch cadence of companies like SpaceX has far surpassed the mark of one launch per week. Furthermore, there is an inherit risk to rocketry and a free-falling rocket can pose major risk to viewers or personnel on site.
Additionally, it was decided to use the next iteration of SPROG's rocket family called ARCHER to aim for the European Rocketry Challenge, which meant that the designs not only had to be improved but also had to be adapted to the rules of EuRoC. This led to the development of ASCENT II, a full redesign of SPROG's first avionics stack.

The objective of this thesis, is to outline the design steps and processes taken for the development of ASCENT II's recovery module called "Electronic Acutation & Guidance for Landing and Eleveation" (EAGLE) whilst staying within the design framework provided by the EuRoC rule book.

1.2 ARCHER

As mentioned before, ARCHER is SPROG's entry for EuRoC. It is a two-staged experimental rocket relying on COTS solid rocket motors as its propulsion system. It will feature a diameter of 160 mm and mostly be made of carbon fiber tubes and aluminum bulkheads. ARCHER's main objective will be to compete in the 9 km solid category and achieve an accurate target altitude. For that it will include an airbrake system, which will control its ascent by deploying flaps similar to SpaceX's grid fins.
At the core of ARCHER's stages sits the modularized avionics stack called ASCENT II. It will serve as the flight computer, meaning that it will deploy the parachutes for both stages and ignite the second stage motor.
ASCENT's design philosophy is based on modules because it makes troubleshooting and expansion of each system easier than having one single avionics board. ASCENT II features a power management system, a sensor module for collecting data, a telemetry module for communication with the ground and EAGLE. The latter deals with parachute deployment, staging, second stage motor ignition and airbrake actuation, all of which are deemed saftey critical systems, because of their potentially dangerous consequences in case of failures.

1.3 Objectives and Scope

The main objective is the theoretical design of the EAGLE PCB and its software architecture, which should accommodate redundancy of systems and be in compliance with EuRoC rules. However, the testing and use of this PCB and its software will not be featured as part of this report. The main goal is to highlight the processes and architecture of EAGLE in the large context of ASCENT II, showing its design iterations and decision making process.
To comply with these objectives, EAGLE will feature a variety of sensors and deal with advanced circuit and software design.

2 Theoretical Basics

To illustrate some of the decisions made within the design phase of EAGLE, it is important to first get a basic understanding of the fundamentals of sounding rocket recovery, avionics architecture and PCB design. This chapter deals with said fundamentals and aims to highlight the most important theoretical topics.

2.1 Electrical Design

Whilst EAGLES's IC circuits are mostly based on datasheets and supplier recommendations, its pyro-channels are fully self-developed. To understand both the IC circuits and the pyro-channels, it is important to understand certain electrical concepts and components, which will be explained in this chapter.

2.1.1 RC-Based Voltage Stabilization of Integrated Circuits

Stable supply voltages are essential for the reliable operation of integrated circuits (ICs). Variations in the supply voltage can occur due to switching activity, sudden changes in load current, electromagnetic interference, and parasitic impedances in the power-distribution network. These disturbances can affect the performance and reliability of an IC. Resistor–capacitor (RC) networks and decoupling capacitors are commonly used to attenuate supply noise and reduce transient voltage fluctuations[1],[2].

A first-order RC low-pass filter consists of a resistor (R) connected in series with the supply and a capacitor (C) connected between the output and ground. Its behavior can be described using Kirchhoff's current law:

$$\frac{v_{\text{in}}(t) - v_{\text{out}}(t)}{R} = C\frac{\mathrm{d}v_{\text{out}}(t)}{\mathrm{d}t} \tag{2.1.1}$$

Applying the Laplace transform under zero initial conditions yields the transfer function:

$$H(s) = \frac{V_{\text{out}}(s)}{V_{\text{in}}(s)} = \frac{1}{1 + sRC} \tag{2.1.2}$$

The cutoff frequency f_c, at which the output amplitude decreases by approximately 3 dB, is given by

$$f_c = \frac{1}{2\pi RC} \tag{2.1.3}$$

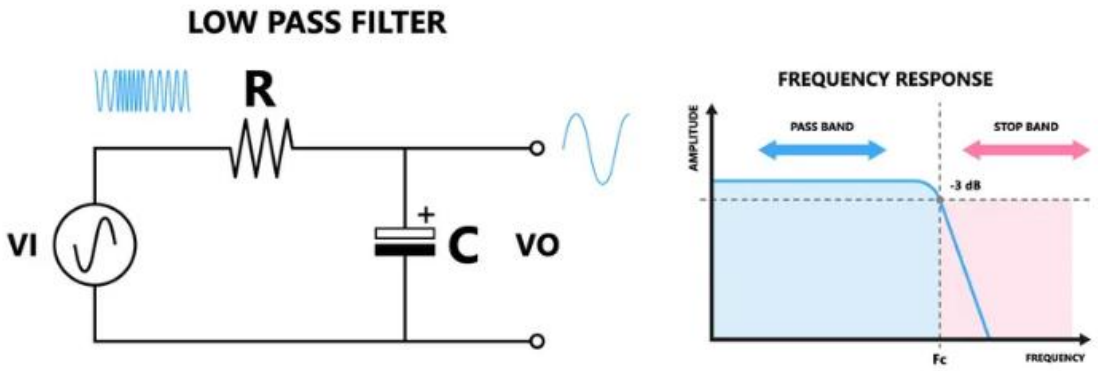


Figure 2.1.1: Typical Low-Pass-Filter circuit and frequency response[3]

Frequencies significantly above the cutoff frequency are attenuated, whereas lower-frequency components pass with relatively little attenuation. Increasing either the resistance or capacitance lowers the cutoff frequency, improving the attenuation of higher-frequency disturbances. However, a series resistor also introduces a DC voltage drop under load and may limit the current available during rapid load changes[4].

The transient behavior of an RC circuit is determined by its time constant, ($\tau = RC$). For a step input from zero to a constant voltage (V_S), the capacitor voltage follows the exponential response

$$v_C(t) = V_S\left(1 - e^{-t/RC}\right) = V_S\left(1 - e^{-t/\tau}\right) \tag{2.1.4}$$

After one time constant, the capacitor reaches approximately 63.2 % of its final voltage, while after five time constants it reaches approximately 99.3 %. This behavior characterizes the response speed of a first-order RC circuit and illustrates the trade-off between filtering and transient response[4].

Decoupling capacitors provide an additional means of stabilizing IC supply voltages. Connected between the supply and ground pins, they act as local charge reservoirs that can supply short-duration increases in current demand. The relationship between capacitor current and voltage is

$$i_C(t) = C\frac{\mathrm{d}v_C(t)}{\mathrm{d}t} \tag{2.1.5}$$

For an approximately constant current demand (ΔI) supplied by the capacitor over a time interval (Δt), the resulting voltage droop can be estimated as

$$\Delta V \approx \frac{\Delta I \Delta t}{C} \tag{2.1.6}$$

This relationship shows that a larger capacitance reduces the voltage deviation for a given current demand and duration. In practical circuits, however, the effectiveness of decoupling also depends on the capacitor's equivalent series resistance (ESR), equivalent series induc-

tance (ESL), and the parasitic impedances of PCB traces and connections. Consequently, decoupling capacitors should be placed close to the relevant IC supply pins to minimize the inductance of the current path. Local ceramic capacitors are often combined with larger bulk capacitors to address transient demands over different time scales, with the final values selected according to the IC datasheet and application requirements[1],[2].

Although RC filters and decoupling capacitors reduce supply noise and transient voltage fluctuations, they do not independently provide precise DC voltage regulation. Their effectiveness depends on the circuit topology, component values, load characteristics, and physical implementation. Therefore, RC-based filtering and decoupling should be considered complementary measures within the power-distribution network, typically used alongside an active voltage regulator to support reliable IC operation[1],[2].

2.1.2 Metal-Oxide-Semiconductor Field-Fffect Transistor (MOSFET)

2.1.3 Inverting Schmitt Trigger

Inverting Schmitt triggers are electronic circuits used to convert slowly varying or noisy input signals into well-defined digital output signals. Furthermore, they can be used to invert a swtiching voltage provided by e.g. a digital output. Unlike conventional inverters, they incorporate hysteresis, meaning that the switching threshold depends on whether the input voltage is increasing or decreasing. This behavior improves noise immunity and prevents unwanted output transitions when the input fluctuates near the switching threshold.

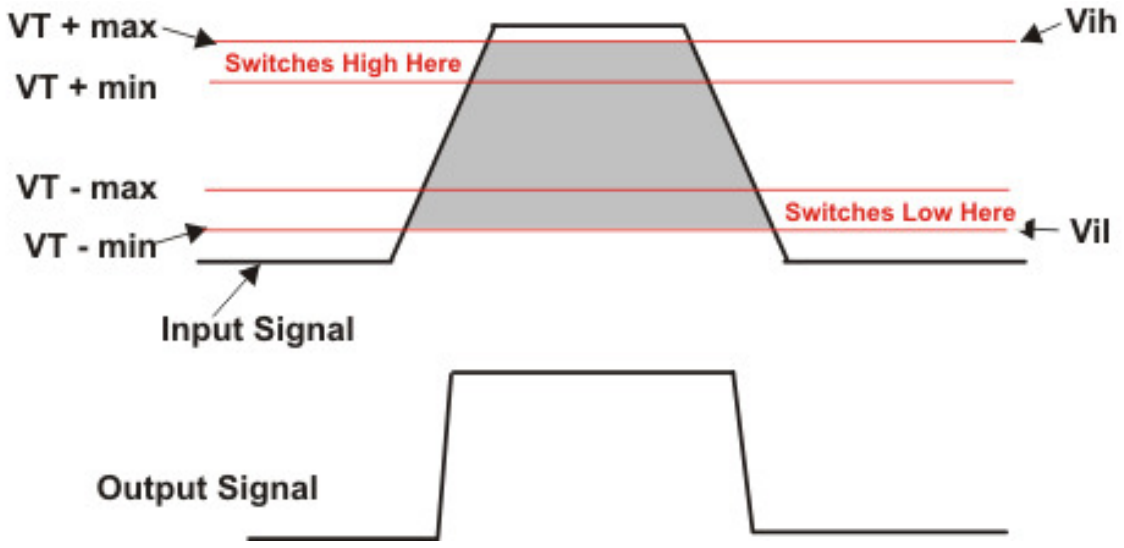


Figure 2.1.2: Typical functionality of inverting Schmitt-Triggers[5]

The operation of an inverting Schmitt trigger is characterized by two threshold voltages: the upper threshold (V_{T+}) and the lower threshold (V_{T-}). When the input voltage rises above (V_{T+}), the output switches from HIGH to LOW. Conversely, when the input voltage falls below (V_{T-}), the output switches from LOW to HIGH. Between these thresholds, the output retains its previous state. The width of the hysteresis window is defined as

$$V_H = V_{T+} - V_{T-} \tag{2.1.7}$$

A larger hysteresis window generally provides greater tolerance to input noise, provided that the input signal still crosses both switching thresholds. The two distinct thresholds prevent small voltage fluctuations from repeatedly triggering the circuit.

In IC applications, Schmitt-trigger inputs are particularly useful for signal conditioning, switch debouncing, and processing signals with slow rise or fall times. They can also be combined with RC networks to create power-on reset delays or controlled signal transitions. In such configurations, the resistor and capacitor determine the rate at which the input voltage changes, while the Schmitt trigger converts this gradual voltage change into a clean digital transition. The resulting behavior depends on the RC time constant, the switching thresholds, and the IC supply voltage.

Although inverting Schmitt triggers improve noise immunity and switching reliability, their threshold voltages and hysteresis characteristics depend on the selected device. Therefore, the manufacturer's datasheet should be consulted when selecting component values or determining timing behavior. Schmitt triggers do not eliminate all noise; disturbances large enough to cross the relevant switching threshold can still change the output state.

2.2 Software Concepts

Any software on potentially dangerous systems has to be maintainable, easily understandable and above all reliable. In order to achieve this, it is important to utilize commonly used software concepts, which have proven to be reliable. Such concepts will be discussed within this chapter.

2.2.1 State Machines

State machines or Finite State Machine are a commonly used concept in software engineering and generally describes a system which can only be in one state of a finite number of state at a time. It is a event-driven and reactive system which can change its state based on certain events happening or conditions that are met.
An example for a state machine would be a video, which can have multiple states such as: Paused, playing, stopped and so on. A possible transition between those states would be something like pressing the pause and play button. Pressing the pause button causes a transition from the state "Playing" to the state "Paused". However, while the video is playing it can not be paused or stopped at the same time.
By only having one state be active at a time, it makes both troubleshooting and designing simpler. This is due to the modularization of the software. If a certain state is being executed, it means that if an issue occurs it can only be caused by this specific state. This instantly narrows down the amount of error sources. Additionally, each state can be developed independently as there are no dependencies between states other than the transition conditions. Each state behaves as if it was its own program that is called from a seperate program. It

also makes the predictability of a system more reliable, because of the same reason. A smaller program is easier to predict in its behaviour by relying on less complexity and dependencies between systems.
The way state machines are typically implemented is by the use of switch-case statements, where each case is representative of a state. Typically, each case or state has its own running loop that executes a certain function, which is executed when its in the right state, e.g. playing the video. Within a loop there would be a breaking condition, e.g. a pause button press event, which would both end the loop and also set the next state, which would then start a different loop.
State machines do not need to follow a linear stream of states. It is common that certain conditions or transitions can cause jumps in the state machine leaving entire states behind. Similarly, it can be possible to go back to a former state instead of moving forward.

2.2.2 PID Controllers

2.3 PCB Design

2.4 Communication Protocols

Embedded use various different common communication protocols such as I^2C, SPI, UART and CAN (not discussed within this thesis). To evaluate the use-case and best fit for EAGLE, it is important to understand the complexity, advantages and disadvantages of each protocol. To achieve said comparison, the protocols I^2C, SPI and UART will be discussed in more detail and lastly compared within this chapter.

2.4.1 Inter-Integrated Circuit (I^2C)[1]

I^2C is a communcation protocol developed by Philips semiconductors in 1982. The acronym, standing for "Inter-Integrated-Circuit" refers to the use case of I^2C, which is to communicate between two or more integrated circuits (ICs) such as a microcontroller and its peripherals such as sensors.
I^2C is an synchronous serial communication bus, which consists of a data line, commonly referred to as "SDA" and a clock signal line, also known as "SCL". It follows the master-slave-principle, meaning that there is a central controller which dictates the clock speed to synchronize the communication over the data line with the peripherals (slaves) to avoid overlapping clock signals. A master can communicate with up to 128 devices on the same data lines by differentiating each slave by an individual address. This means that two slaves can never have the same address to avoid collision because all slaves are connected to the same data lines.
Communication between master and slave is initiated by broadcasting the address of the slave device, after which the intent of the master (whether it wants to read or write) is communicated to the selected peripheral. Data is only sent or read after intent and address have been established. I^2C has different speed rates for the communication rates: Standard,

[1] The information contained within this chapter on I^2C has been taken from the document "I2C-Bus" by Ramesh Sathiyamoorthy[6], which was uploaded to the online forum mikrocontroller.net

Fast and High-speed mode. The maximum data rate can be as high as 3.4 MBit/s.
To establish communication, the SDA line is pulled low by the SDA bus, while the SCL bus remains high. This implies, that I^2C requires to have a high voltage level by default. This is done by connecting SDA and SCL to the supply voltage (or IO voltage) through pull-up resistors of a generic value (e.g. 10 kΩ). The specific sizing of the pull-up resistors are usually defined or recommended in the slave manufacturer's datasheet. During active communication, data is transmitted byte-wise and is synchronized by the clock speed. Communication comes to a halt after a rising edge on the SDA bus has been detected, while the SCL line is also high. This implies, that SCL and SDA may never be HIGH at the same time as it would trigger this stop sequence.
One of the major downsides of the I^2C is that only one device can be called at a time, meaning that devices would have to be called sequentially. Furthermore, there may only be on slave per I^2C bus, which in comparison to communication protocols such as CAN (will not be discussed further), is not suitable for communications with multiple microcontrollers that may need to be established as masters themselves (e.g. communications between subsystems which each have their own sensors).

2.4.2 Universal Asynchronous Receiver-Transmitter (UART)[2]

Universal Asynchronous Receiver-Transmitter, better known as "UART", is a serial communication protocol which transmits data asynchronously, meaning that data does not follow a common clock signal, but sending and receiving can happen at different times. Communication via UART can be simplex (one-way communication), half-duplex (both-way communication, one communication mode at a time) or full-duplex (both-way communication, both modes at the same time).
Usage of UART has decreased over the last years with the establishment of faster serial communication protocols such as I^2C and SPI. However, most microcontrollers (e.g. ESP32 or Arudinos) still implement UART as a bus to flash new software via a USB to UART converters, but it can also be used to communicate with external devices that use e.g. an RS-232 interface for communication.
UART uses two buses commonly referred to as "TX" (Transmitter) and "RX" (Receiver). Since UART does not establish a master, all connected devices have the same authority and priority on a bus, meaning that the UART pins have to be inverted on the other end of the communication path. This means that the RX of the first device has to go to the TX of the other device and vice-versa for the TX of the first device. Additionally, only two devices can communicate over the same two buses, because UART does not have a way to differentiate between devices like I^2C or SPI. It is possible to have multiple devices listening to the same transmissions, but it is not possible to write to the same line. Since the transmitter and receiver do not share a common clock signal, both devices have to be initialized to the same baud/data rate (commonly 9600 baud) in order to allow reading and writing at the same speed. To send data UART uses data frames, which contain start and stop bits as well as the data bits and a parity bit as seen in figure 2.4.1.

[2]The information contained within this chapter on UART is based on the online article "Understanding UART" by the company Rhode & Schwarz[7]

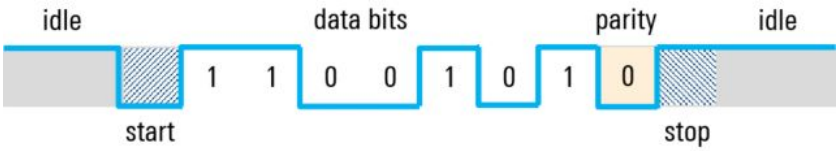


Figure 2.4.1: Typical UART data frame structure[7]

Since UART does not follow a master-slave-principle communication is not established by announcement to the slave, which is why the communication is initiated by a start bit that tells the other bus that there is data coming in. The stop bit, in turn, tells the other device that the data frame has been fully transmitted. The actual data bits (user data) is up to 9 bits long (commonly 7 to 8 bits). At the end of the user data, an optional parity bit can be inserted that can be used for error detection. It relays if the total number of 1's in the data frame has been odd or even, which can indicate an error, if the received data has a different parity.

2.4.3 Serial Peripheral Interface (SPI)[3]

The Serial Peripheral Interface (SPI) is a synchronous serial communication protocol developed by Motorola in 1985. SPI's main use-case lies in short communication path applications such as connection between microprocessors and peripherals such as sensors. Similarly to I^2C, SPI uses a master-slave-principle at full-duplex, but instead of using a addresses to differentiate between its slaves, it uses a hardware select pin to choose the peripheral or slave it wants to communicate with, with clock speeds of up to 10 MHz.
SPI uses 4 pins for communication. For data transmitting and receiving, it uses the MOSI (Master Out Slave In) and MISO (Master In Slave Out) pins, which, as the name suggests) are either the receiving pin or the transmitting pin (seen from the master). As already mentioned, SPI is a synchronous protocol, which means communication follows a common clock signal, which is provided by the SCK (clock) pin. Lastly, the slave needs to provide a chip select (CS) pin to select the slave it wants to communicate with.
Because SPI uses a chip select that determines the communication length, the transmitted byte stream can be as long as needed, as the only thing terminating or starting the communication between the master and slave is said CS pin. If said pin is not changed, the state of the communication remains the same.
If communication with multiple slaves is not desired, a digital output has to be used as a CS pin for each individual peripheral. This is particularly important if the communication is not

[3]The information on SPI within this chapter has been gotten from the online article "Why and How to Use the Serial Peripheral Interface to Simplify Connections Between Multiple Devices" written by Art Pini[8] and published on DigiKey's forum

one-way. If two peripherals were "selected", both would try to send data to the same data line, which would overlap the signals and cause the data to be corrupted. SPI devices are usually "active low", meaning that the CS pins are usually kept high and are pulled low to "select" a slave. Similarly to I^2C, the data lines MISO and MOSI also need pull-up resistors (typically 10 kΩ). To top communication, the CS pin is then put back to the high value.

2.4.4 Comparison

I^2C, UART and SPI are among the most commonly used communication protocols in embedded systems and have established themselves as the standard for communication with sensors and similar peripherals. However, each protocol comes with its own set of advantaged and disadvantages.

Table 2.4.1: Comparison of communication protocols

Protocol	**Available modes**	**Number of pins needed**	**Devices per interface[4]**	**Max. frequency**
I^2C	Half-Duplex	2	128	3.4 MBit/s
UART	Simplex Half-Duplex Full-Duplex	2	1	N/A[5]
SPI	Simplex Half-Duplex Full-Duplex	Min. 4	∞[6]	10 MHz

As seen in table 2.4.1 I^2C only allows half-duplex, whereas the other devices allow for every communication mode. However, it does allow up to 128 devices on just two pins (given that every device has an individual address), which is very cheap when considering the amount of peripherals could exceed ten. For that reason, UART is only really reasonable for communication that does not require a master and slave, such as the communication between two microcontrollers. Additionally, SPI quickly becomes a hardware bottleneck as the amount of digital outputs needed for CS might exceed the available or needed pins for other tasks. On the other hand, SPI can be used for higher polling speeds, if the required rate exceeds I^2C's maximum frequency of 3.4 MBit/s. In summary, I^2C is the most versatile when it comes to having many slaves at the same time e.g. in a system with many sensors. SPI becomes more reasonable for higher communication speeds and UART can be used for communication between e.g. two microcontrollers as it does not establish a master-slave-relationship.

2.5 Sounding Rocket Recovery Systems

The term "sounding rocket" refers to a suborbital class rocket, which usually only goes to hundreds of kilometers in altitude in order to reach space and achieve a certain amount of zero gravity. This results in the vehicle falling back down to earth after a short amount of time in space. Every sounding rocket features some form of recovery since a free-falling rocket at speeds much higher than the speed of sound poses too much of a risk for nearby people

[4]Only including peripherals
[5]Hardware reliant, 115.2 kBaud standard
[6]Limited by amount of digital outputs available as CS pins

and personnel. Whilst modern orbital class rockets are either expendable (e.g. Ariane 6) or recovered using propulsive landing methods (e.g. SpaceX Falcon 9), sounding rockets usually rely on parachutes. This is due to the fact, that the main goal of sounding rockets is to provide a short time in free-fall or zero gravity for experiments[9], which could otherwise not be completed within earth's atmosphere. This means that the main goal is to maximize payload capacity and minimize the complexity and cost. This is usually achieved by relying on a "simpler" architecture than bi-liquid rockets, such as hybrid or solid motors. For simplicity, only solid motors will be discussed.

A solid motor consists of a mix of the fuel and oxidizer in one solid so called "grain", which is ignited and burns at a more or less constant rate. This results in an uncontrolled thrust-curve which is controller by the motors geometry and chemical composition. This makes a complicated system such as tanks and pressurized systems mostly obsolete, because fuel and oxidizer are both stored in the same space. This vastly reduces wait and space taken up by the propulsion segment. Since it is not possible to control thrust on in a solid booster, the recovery can not be made using propulsive methods, which is why parachutes are used.

Most recovery systems consist of more than one parachute, the so called "drogue-parachute" and "main-parachute". The drogue is usually significantly smaller than the main parachute. For smaller sounding rockets that do not cross into space, the reason for that is that the drogue is usually deployed at/or around apogee to stabilize the rocket (as shown in fig. 2.5.1), position it vertical and start initial deceleration[10]. The main parachute is only deployed shortly above the surface. Due to the much bigger surface area of the main parachute, the amount of lift created is substantially higher, resulting in a bigger deceleration and slower descent. However, because of said lift and slower descent, the rocket has more time to drift down range, meaning that it lands further away from the launch pad, which makes the recovery after touchdown much more difficult as there is a bigger potential landing area.

When the drogue is deployed, it is usually still attached to a piece of rope that holds it close to the air frame of the rocket and the main parachute. When the main parachute is deployed, said rope is cut or otherwise disconnected and the drogue creates a bigger distance due to its wind resistance. In doing so, it pulls out the main parachute which then inflates and decelerates the rocket to its dedicated landing velocity[10].

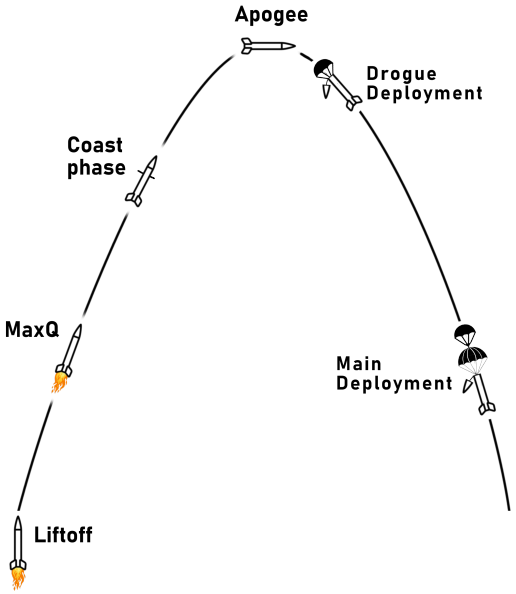


Figure 2.5.1: Idealized trajectory of a sounding rocket

2.6 Flight Trajectory[7]

Because a sounding rocket follows a ballistic, parabolic flight path, its ascent can be divided into a series of distinct phases. Several of these phases mark "milestones" that are directly tied to specific events during flight, such as parachute deployment or the separation of stages in a multi-stage vehicle.
Eight milestones are relevant in this context which are shown in figure 2.5.1: Liftoff, MaxQ, booster burnout, the coast phase, apogee, drogue deployment and main parachute deployment. Each is described below.
The vehicle experiences its highest g-forces at liftoff, the moment it leaves the launch rail immediately following booster ignition. Because amateur rockets tend to have a very high thrust-to-weight ratio - large upward thrust paired with comparatively little mass - liftoff produces a pronounced jolt that puts considerable strain on the rocket's components. This spike in acceleration can be used to detect liftoff by accessing data from the accelerometer and IMU of the vehicle's avionics system.
A short time later, the vehicle reaches its point of maximum dynamic pressure, known as "MaxQ." At this moment, aerodynamic loading on the vehicle peaks, and with it the mechanical stress caused by aerodynamically induced bending moments on the structure - making MaxQ the point of greatest aerodynamic force during the entire flight.
Once the motor has burned almost completely, the booster reaches what is called burnout. A small flame may still be visible afterward, as residual propellant continues to burn off, but the thrust it produces at this stage is negligible. What defines burnout, more precisely, is the moment the booster loses the bulk of its thrust, resulting in a acceleration that is similar to

[7]The following theoretical foundations on the topic of flight trajectories of suborbital rockets are based in part on the author's previous work from the student project on the topic of the mechanical design of EAGLE (see "Mechanical Design of Electronic Actuation & Guidance for Landing and Elevation (EAGLE) system"[11]). This approach is expanded upon below to include the aspects parachute deployment in order to elaborate the milestones which EAGLE will have to be able to autonomously detect

earth's gravitation of $\approx 9.81\,\frac{\mathrm{m}}{\mathrm{s}^2}$ (relating to the axis perpendicular to the ground), which can serve as indication for autonomous detection of burnout.
Between booster burnout and apogee lies the coast phase. No longer driven by the comparatively unpredictable thrust of a solid motor, the vehicle is now governed only by gravity and aerodynamic forces during this stretch of the flight. As a result, its behavior becomes highly deterministic and predictable - making the coast phase the ideal window in which to apply aerodynamic control.
The flight culminates at apogee, the highest point of the trajectory and the control setpoint for EAGLE. Vertical velocity falls to nearly $0\,\frac{\mathrm{m}}{\mathrm{s}}$ at this moment, and the rocket transitions into free fall. This point is typically used as a trigger of the deployment mechanism (e.g. pyrotechnic charges) of the drogue parachute.
From this point, the rocket falls at a constant speed, which is determined by the size and lift of the drogue parachute. After reaching the target setpoint for main parachute deployment, another mechanism is triggered (e.g. a rope cutter) that causes the drogue parachute to disconnect from the airframe of the rocket such that it only remains connected to the main parachute. This results in the parachute being dragged out and therefore deployed. Afterwards, the rocket glides down at a constant final speed, which will also be the speed with which it touches down on the ground.

2.7 Barometric Formula

The barometric formula can be used to correlate atmospheric pressure, temperature or air density to the altitude above sea-level. It is used in aircraft and lower flying sounding rockets to estimate its altitude. This is done by measuring the surrounding absolute air pressure and calculating the resulting altitude.

2.7.1 Hydrostatic equation

The underlying mathematical mechanics of the barometric lie in the hydrostatic equation, which describes the correlation between pressure at a defined depth within a fluid.
To understand the basic concept of hydrostatics, a infinitesimal volume element $\mathrm{d}V$ of base area A and infinitesimal height $\mathrm{d}h$ can be considered. The force F exercised on the base area at a known pressure p of a fluid of density ρ can be described as:

$$F_{\mathrm{hydro}} = p \cdot A \qquad \text{with } p = \rho g h \tag{2.7.1}$$

$$= \rho g h \cdot A = \rho g \cdot V \qquad \text{with } m = \rho V \tag{2.7.2}$$

$$F_{\mathrm{hydro}} = m \cdot g = \rho g \cdot V \tag{2.7.3}$$

When considering the volume to be infinitesimal, the function can be described as follows:

$$\mathrm{d}m \cdot g = \rho \cdot \mathrm{d}V = \rho g \cdot \mathrm{d}h \cdot A \tag{2.7.4}$$

Based on equation 2.7.4, it is possible to calculate the Force F exercised on volume element $\mathrm{d}V$:

$$F_{\text{hydro}}(h+\mathrm{d}h) = F_{\text{hydro}}(h+\mathrm{d}h) + F_{\text{hydro}}(h) \qquad \text{with eqn. 2.7.2} \tag{2.7.5}$$

$$= \rho g(h+\mathrm{d}h)\cdot A + \rho g h \cdot A = A(\rho g(h+\mathrm{d}h) + \rho g h) \qquad \text{with } p = \rho g h \tag{2.7.6}$$

$$F_{\text{hydro}}(h+\mathrm{d}h) = A(p+\mathrm{d}p) \tag{2.7.7}$$

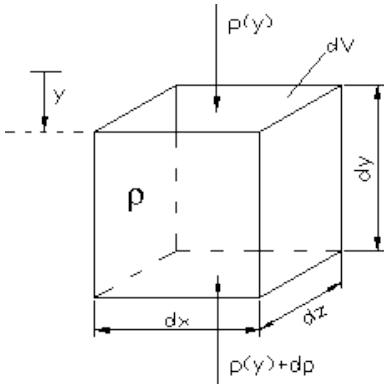


Figure 2.7.1: Volume element in hydrostatic system[12]

2.7.2 Isothermal barometric formula

When considering the system to be in equilibrium, meaning that all acting forces cancel each other out, the system can be described as follows:

$$F_{\text{G}} = F_{\text{hydro}}(h) - F_{\text{hydro}}(h+dh) \tag{2.7.8}$$

$$0 = F_{\text{hydro}}(h) - F_{\text{hydro}}(h+dh) - F_{\text{G}} \qquad \text{with } F_{\text{G}} = \rho g \cdot \mathrm{d}h \cdot A \tag{2.7.9}$$

$$0 = pA - A(p+dp) - \rho g \cdot \mathrm{d}h \cdot A \tag{2.7.10}$$

$$\frac{\mathrm{d}p}{\mathrm{d}h} = \rho g \tag{2.7.11}$$

When considering this system, we assume the fluid to be incompressible and follow the laws of an ideal gas. This means that the gas molecules are considered to be mass points of no volume, there are no acting intermolecular forces and the collisions are considered elastic. By doing so we can use the formular for density of an ideal gas:

$$\rho = \frac{pM}{RT} \tag{2.7.12}$$

M represents the molar mass, so the mass per $6.022 \cdot 10^{23}$ (1 mol) particles. R stands for the universal gas constant, which is the equivalent of $8.314\,\frac{\text{J}}{\text{K}\cdot\text{mol}}$ and T being the absolute temperature in K. By substituting ρ from equation 2.7.11 with equation 2.7.12 and integrating over the pressure and height differential, you get the following expression:

$$\frac{\mathrm{d}p}{\mathrm{d}h} = \rho g \tag{2.7.13}$$

$$\frac{\mathrm{d}p}{\mathrm{d}h} = -\frac{pM}{RT} g \tag{2.7.14}$$

$$\frac{\mathrm{d}p}{p} = -\frac{Mg}{RT} \mathrm{d}h \tag{2.7.15}$$

$$\int_{p(h_0)}^{p(h_1)} \frac{1}{p}\,\mathrm{d}p = -\int_{h_0}^{h_1} \frac{Mg}{RT}\,\mathrm{d}h \tag{2.7.16}$$

$$\ln\frac{p(h_1)}{p(h_0)}) = -\frac{Mg}{RT}(h_1 - h_0) = -\frac{Mg}{RT}\Delta h \tag{2.7.17}$$

$$\frac{p(h_1)}{p(h_0)} = e^{-\frac{Mg}{RT}\Delta h} \tag{2.7.18}$$

$$p(h_1) = p(h_0) e^{-\frac{Mg}{RT}\Delta h} \tag{2.7.19}$$

2.7.3 Adiabatic barometric formula

Equation 2.7.19 is derived under the assumption that the temperature stays constant with rising altitude. However, the temperature changes with the altitude, which makes this isothermal formula inaccurate for altitudes over $\approx$5 km.
For simplicity reasons, the atmosphere will be assumed as adiabatic, meaning that no energy will be exchanged with external systems, meaning the internal energy of the system will remain constant. By doing so, we can assume the system to be an adiabatic process following the laws of the following equation:

$$\frac{\mathrm{d}p}{p} = \frac{\kappa}{\kappa - 1}\frac{\mathrm{d}T}{T} \tag{2.7.20}$$

κ is the "heat capacity ratio" and describes the ratio between the heat capacity of a gas at constant pressure and a gas at constant temperature. For air, κ can be assumed to 1.402. Using equation 2.7.15, the following temperature gradient can be calculated:

$$\frac{\mathrm{d}p}{p} = -\frac{Mg}{RT} dh \qquad \text{with } \frac{\mathrm{d}p}{p} = \frac{\kappa}{\kappa - 1}\frac{\mathrm{d}T}{T} \tag{2.7.21}$$

$$\frac{\kappa}{\kappa - 1}\frac{\mathrm{d}T}{T} = -\frac{Mg}{RT}\mathrm{d}h \tag{2.7.22}$$

$$\mathrm{d}T = -\frac{\kappa - 1}{\kappa}\frac{Mg}{R}\mathrm{d}h \tag{2.7.23}$$

By integrating over the height an temperature, the following equation for the temperature vs. height correlation can be calculated:

$$\int_{T(h_0)}^{T(h_1)} \mathrm{d}T = -\int_{h_0}^{h_1} \frac{\kappa-1}{\kappa}\frac{Mg}{R}\,\mathrm{d}h \tag{2.7.24}$$

$$T(h_1) - T(h_0) = -\frac{\kappa-1}{\kappa}\frac{Mg}{R}\Delta h \tag{2.7.25}$$

$$T(h_1) = T(h_0) - \frac{\kappa-1}{\kappa}\frac{Mg}{R}\Delta h = T(H_0)\left(1 - \frac{\frac{\kappa-1}{\kappa}\frac{Mg}{R}\Delta h}{T(h_0)}\right) \tag{2.7.26}$$

The resulting equation describes the temperature gradient within the atmosphere under the above mentioned assumptions. By substituting the temperature in equation 2.7.15 and once more integrating over the altitude and pressure, the adiabatic pressure gradient in regards to the altitude can be derived:

$$\frac{\mathrm{d}p}{p} = -\frac{Mg}{RT}\mathrm{d}h \tag{2.7.27}$$

$$\frac{\mathrm{d}p}{p} = -\frac{Mg}{RT(H_0)\left(1 - \frac{\frac{\kappa-1}{\kappa}\frac{Mg}{R}\Delta h}{T(h_0)}\right)}\mathrm{d}h \tag{2.7.28}$$

$$\int_{p(h_0)}^{p(h_1)} \frac{1}{p}\,\mathrm{d}p = -\int_{h_0}^{h_1} \frac{Mg}{RT(h_0)\left(1 - \frac{\frac{\kappa-1}{\kappa}\frac{Mg}{R}\Delta h}{T(h_0)}\right)}\mathrm{d}h \tag{2.7.29}$$

$$p(h_1) = p(h_0)\left(1 - \frac{\kappa-1}{\kappa}\frac{Mg\Delta h}{RT(h_0)}\right)^{\frac{\kappa}{\kappa-1}} \tag{2.7.30}$$

2.7.4 International barometric formula

The final equation can be simplified using the standard atmosphere, which is defined by the "International Civil Aviation Organization" (IACO). The standard atmosphere defines sea level as reference height h_0, 15 °C or 288.15 K as the reference temperature $T(h_0)$ and 1013.25 hPa as the reference pressure $p(h_0)$[13]. This results in the final formula of both the pressure at altitude h and the altitude itself being:

$$p(h) = 1013.25 \cdot \left(1 - \frac{0.0065\,\frac{\mathrm{K}}{\mathrm{m}} \cdot h}{288.15\,\mathrm{K}}\right)^{5.255}\,\mathrm{hPa} \tag{2.7.31}$$

$$h = \frac{288.15\,\mathrm{K}}{0.0065\,\frac{\mathrm{K}}{\mathrm{m}}}\left(1 - \left(\frac{p(h)}{1013.25\,\mathrm{hPa}}\right)^{\frac{1}{5.255}}\right) \tag{2.7.32}$$

2.8 Nyquist Criteria

2.9 Avionics Architecture for Recovery Systems

3 Background and Related Work

Avionics systems on student rockets is not a solved issue, as teams come up with new innovative ideas and concepts to increase performance and optimize processes. To put the development of EAGLE into perspective and elaborate the importance of SRAD technology, this chapter highlights already established work and related topics in regards to industrial solutions and development within student teams.

3.1 DLR - MORABA

MORABA, the Mobile Rocket Base, is a department of the German Aerospace Center (DLR) Space Operations and Astronaut Training institute, located in Oberpfaffenhofen near Munich. Founded in 1968 as a mobile launch service unit, it has become one of the leading European organizations for the preparation and execution of sounding rocket and balloon campaigns. Its mobility is a defining feature: the entire launch infrastructure, including launchers, telemetry and tracking stations, and range safety equipment, can be transported and set up at remote sites worldwide, from Esrange in Sweden and Andøya in Norway to Kourou, Woomera, and Arctic and equatorial locations.
Beyond campaign operations, MORABA provides the full range of services required for a sounding rocket mission. These include mission analysis and trajectory simulation, vehicle design and integration, flight dynamics and guidance, navigation and control, structural and thermal analysis, as well as payload and recovery system development. MORABA operates a fleet of solid-propellant rockets, such as VSB-30, Improved Orion, and Brazilian-developed vehicles, and has additionally developed its own systems, including the Rocket Attitude Control System (ACS) for precise pointing during microgravity and astronomy missions.
Over the decades, MORABA has supported hundreds of flights for scientific research in microgravity, atmospheric physics, and astronomy, as well as technology demonstration missions, including hypersonic and re-entry experiments and reusable launch vehicle testing (e.g., the DLR-led ReFEx experiment)[14].
MORABA employs a typical flight profile of a two-staged sounding rocket as shown in figure 3.1.1. Similarly to EAGLE, MORABA's flight profile also features notable milestones such as stage seperation and second stage booster ignition as well as drogue and main parachute deployment. However, a major difference for parachute deployment is the altitude at which it reached/executed. Different to many student rockets, which only rise to an apogee of tens of kilometers, MORABA's high altitude rockets, which go as high as 800 km do not deploy their drogue parachute at apogee. This is due to the fact, that the higher altitude brings many new challenges, such as the aero-thermodynamical effects of reentry. During reentry, the thermal stress and dynamic pressure on the vehicle prevent a deployment of the drogue parachute as it would not be able to survive this phase without burning up. Nevertheless, MORABA's

avionics detect similar event and execute similar commands as SPROG's avionics. They need to detect first stage booster burnout, ignite a second stage and deploy parachute based on sensor readings.

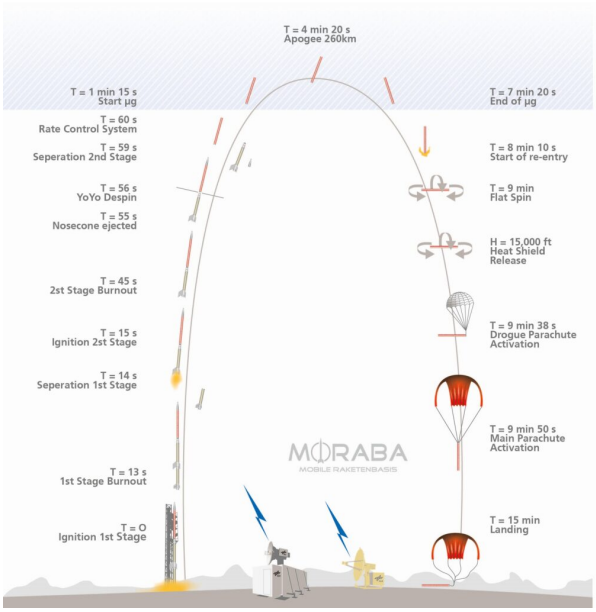


Figure 3.1.1: Trajectory of typical two-staged sounding rocket[14]

3.2 European Student Rocketry

Student rocketry in Germany has developed over the past two decades from small, hobby-scale initiatives into a structured and technically ambitious research and education landscape. Driven by university-based teams, often organized as interdisciplinary student groups, these projects give students hands-on experience in the complete development cycle of a launch vehicle, from requirements definition and design to manufacturing, testing, and flight operations. Groups such as WARR (Wissenschaftliche Arbeitsgemeinschaft für Raketentechnik und Raumfahrt) at the Technical University of Munich (TUM), founded in 1962, and other teams at universities including Stuttgart (HyEnd), Aachen (Space Team Aachen), and Braunschweig (ERIG), have established long-standing traditions and a growing body of technical know-how. Furthermore, there has been an additional increase of student teams in other parts of Germany and Europe, SPROG among them.

A central characteristic of the German scene is its progression from solid-propellant vehicles toward more complex systems, including hybrid and liquid-propellant engines, active guidance and control, and recovery systems. Several teams have pursued altitude records and technology demonstrations, and projects such as the hybrid rockets developed at the University of Stuttgart and the liquid-propellant developments at TUM have gained international attention. Students also increasingly work on subsystems such as avionics, telemetry, in-house engine test stands, and flight software, which mirrors professional aerospace workflows.

Institutional support plays an important role. Programs such as DLR's REXUS/BEXUS (Rocket and Balloon Experiments for University Students), carried out in cooperation with the Swedish National Space Agency (SNSA) and ESA, and implemented with the support of MORABA, allow student experiments to fly on real sounding rockets and balloons launched

from Esrange. Together with events such as the European Rocketry Challenge (EuRoC) in Portugal and the Spaceport America Cup in the USA, these opportunities provide German teams with launch access, technical review processes, and international benchmarking. Student rocketry thus forms an important bridge between academic education and the professional sounding rocket community, and it provides the context for the work presented in this thesis.
Additionally, student rocketry plays a major role in the future of the european launcher industry. This is largely due to the fact that student teams not only teach the necessary skills to develop comprehensive technology, but also encourages the development of startups. The most recent example has been the NewSpace launcher company "Isar Aerospace", who have recently been able to achieve an orbit on only their second flight. Isar Aerospace's founders have previously been part of WARR, where the idea was supported through the "UnternehmerTUM Accelerator" program[15] further establishing the importance of student teams.

3.2.1 Bundesverband studentischer Raumfahrt (BVSR)

The "Bundesverband studentischer Raumfahrt e.V." (engl. "German National Student Spaceflight Society") is national umbrella organization with 19 (corresponding[1]) member associations dedicated to connect the industry with student rocket associations. Its goal is to play a significant part in the future development of the european space sector by 2035[16].
Each year, one of BVSR's member clubs hosts the yearly BVSR conference where all associations get to meet up, connect with each other and decide on important topics regarding the next year of the BVSR. Additionally, it features multi-disciplinary workshops and fair including major space sector players such as OHB, Airbus and etc. underlying the importance of the student sector from the eyes of major space corporations.
On top of that, the BVSR actively engages in its own research and projects such as ODIN, a design competition aimed at designing a sub-scale orbital class rocket as part of a competition between student groups[16]. Entries will be judged by a juror board consisting of industry experts. They are also cooperating with the "Starlab Initiative" to explore possible experiments and instrumentation of the commercial space station called "Starlab"[16], which many believe to be the successor of the Internation Space Station.
To summarize, the BVSR already plays a large part in shaping the space sector in Germany and in Europe with engagement both on political and technical level.

3.2.2 European Rocketry Challenge

The "European Rocketry Challenge" (EuRoC) is an international student rocketry competition hosted and organized by the Portugese Space Agency which was first introduced in 2020. Each year a selection of student teams across ESA member states compete in different categories such as altitudes (3 and 9 km) or propulsion technologies (solid, hybrid and bi-liquid). Especially propulsion technologies get extra attention in the so called "SRAD" (student researched and developed) classed categories as they showcase the innovation and engineering

[1] Student associations outside of Germany that are not full members

skills of students with limited funding. This competition has pushed teams to challenge the limits of student rocketry by building e.g. the first ever student-built regeneratively cooled bi-liquid engine in Europe by WARR Rocketry.
The competition awards multiple winners in different categories. Awards are decided based on the design, team spirit and accuracy regarding the targeted altitude. A team that achieves an apogee close to their target altitude receives more points than a team that is far away.
It provides an ideal environment to prove skills under high stress environments for young and aspiring aerospace students whilst also connecting teams from across Europe. The aim is to strengthen Europe's aerospace sector by promoting a connected education program through friendly competitions.

4 Requirement Analysis

With ASCENT being SPROG's main flight computer for in-flight decisions during EuRoC, it does not only have to comply with internal requirements, but also external requirements set within the official sytsem design rule book provided by the portugese space agency. Failing to comply with set rules will lead to either a failed launch readiness review ahead of the launch during competition days or exclude any chance of qualification.
Additionally, SPROG's own vision on the architecture and redundancy also plays a huge factor. The requirements stated by both parties will be discussed in this chapter.

4.1 Control Output and Power Requirements

The recovery team opted for a pyrotechnic charge to deploy the drogue parachute, meaning that EAGLE would need to be able to fire pyrotechnic igniters. During PIPE's launch campaign, it was noted that having the firing electronics external to the PCB leads to a multitude of issues including wiring and space management. That is why the requirement was made to add all electronics having to do with pyrotechnics onto the PCB. Similarly, it was also decided to not use a screw terminal to fixate the igniters, EAGLE will sit in the middle of the avionics stack. This would make replacement of the igniter require a full de-stacking of the avionics system, which is both time consuming and complex. It would therefore be required to be able to replace the igniters in a full stack, meaning that the connector has to be chosen accordingly. The igniters that were chosen also required a current of at least 1 A to fire. This means EAGLE needs to be able to provide 1 A on all 4 channels, ideally simultaneously.
Additionally, EAGLE needed to be able to provide "power width modulation" (PWM) signals to control the airbrake's servo. However, EAGLE does not need to provide the power to either of these systems. The power needed would be provided by another avionics module according to EAGLE's power requirements. The power provided to each PCB of the stack is limited to 3.3 V and 5 V. For the igniters and additional V_{pyro} of 12 V will be provided. Therefore, no component can use more than 5 V as its supply voltage.

4.2 Sensor Requirements

Due to the system additions made between PIPE and ARCHER, EAGLE will need to be able to handle a multitude of tasks reliably and accurately. Before even considering an application for EuRoC it was decided that EAGLE would be put in charge of recovery, staging and airbrake control. Since said tasks require strong reliability and accuracy, it was decided that EAGLE itself should be a dual redundant system featuring at least two sources for each data type (e.g. pressure, acceleration, etc.) that would be needed to accurately determine the

rockets position and attitude. To achieve this, it was decided to determine the altitude using a processed value of acceleration data (through integration) and barometric pressure (altitude calculation). Furthermore, the acceleration data and barometric altitude estimation could be used to estimate the rockets speed. For this reason, two entirely different pressure sensors and accelerometers were selected. On top of that, an inertial measurement unit (IMU) was selected to provide feedback on the rotation speed of the rocket during ascent and additional acceleration measurements.
Additionally, all sensor will have to be I2C compatible at a clock speed of at least 1 kHz. A sampling rate of at least 1 kHz ensures that there is real-time access to sensor data, which determines an in-flight decision resolution of around 2 ms, according to the nyquist criteria. This however, is not a hard requirement regarding the loop time as a mechanical system themselves have a reaction time larger than the loop time. Therefore, it does not need to be accurate to the next ms. The reason I2C was chosen is because of its lower complexity in comparison to SPI. SPI requires a chip select pin for each peripheral, which would take to many output pins away from more important tasks. This comes at the downside of the half-duplex nature of I2C and consequent lower communication speeds.

4.3 EuRoC system requirements

Due to EAGLE wide-range of applications, it becomes necessary to apply multiple requirements set by the organizer's across the following categories:

- Ignition systems
- Recovery systems and avionics
- Active flight control systems
- (Trajectory and stability)

The latter will not be discussed within this report, as it has already been discussed in an associated project showing the design of the mechanical aspect of EAGLE (i.e. the air-brakes[11]), where such topics have been discussed. On top of that, requirements that do not directly influence the design of EAGLE will not be discussed.

4.3.1 Ignition Systems

According to the design requirements ignition systems shall be disarmable and only be armed upon detecting liftoff [17, p. 16]. In this case, arming means that there needs to be at least two actions required for any igniter firing. EAGLE pyrotechnic firing channels must therefore be able to be armed both through soft- and hardware.
As already specified by SPROG's recovery team, EuRoC also requires an electronic ignition system for all solid rocket motor ignitions [17, p. 15]. This implicates that EAGLE requires the before mentioned electronic firing channels.

4.3.2 Recovery Systems and Avionics

The before mentioned ignition requirements also have some implications for the recovery system and avionics section. EuRoC requires a fully redundant recovery system including seperate flight computers and sensor [17, p. 24], including a COTS flight computer. This also includes all parts involved in the electric firing of pyrotechnic charges. As already discussed, two sensors per data type has already been required by SPROG and has been further confirmed by the organizer's rules.
For two staged vehicles, it is mandatory to employ a dual deployment recovery consisting of a drogue and main parachute [17, p. 28] for every part of the rocket that is estimated to reach an apogee of more than 450 m. This paired with a fully redundant pyrotechnic firing channels means that there needs to be at least four channels per stage (2 per parachtue). Additionally, initial deployment should happen "at or near apogee"[17, p. 28], meaning that EAGLE's software will need to utilize the data to accuractely detect apogee. Furthermore, main deployment "shall occur at an altitude no higher than 450 m above ground level"[17, p. 29]. This requires an accurate measurement of the altitude through the software data evaluation.

4.3.3 Active flight control systems

EuRoC limits use of active flight controls of the rocket itself to aerodynamic braking, pitch and roll stability augmentation [17, p. 45]. ARCHER utelizes aerodynamic braking and therefore falls under these requirements. Similarly to the recovery, all electronics associated with the controls will have to be fully redundant[17, p. 47], meaning that more than one PWM signal will have to be provided in case of failure of one channel. On top of that, the control system will need to have an adjustable dormant phase, during which the system can not be used[17, p. 46-47], which can be implemented both in software and in hardware (only the software aspect will be discussed).

5 System Architecture

It is important to understand the architecture around EAGLE and ASCENT II in order to understand some of the decisions in a bigger context. As shown in chapter 4, both hard- and software play a crucial part in understanding how EAGLE works and works it (un-)reliable. Therefore, this chapter aims to outline the overall architecture of ASCENT II and EAGLE. In doing so, both the hardware and software will be discussed.

5.1 Hardware Architecture

EAGLE's own hardware is heavily reliant on external inputs from the other avionics modules. Whilst it is as self-reliant as possible, having a fully standalone system defeats both the purpose of redundany as well as a modularized avionics stack. Therefore, it is important to discuss both the overall architecture and the architecture relating to EAGLE itself.

5.1.1 ASCENT II Architecture

ASCENT II consists of four modules: Telemetry, sensor systems, EAGLE/Recovery and the energy management system as shown in figure 5.1.1. Each subsystem in the flight computer serves an individual purpose and all components interact with each other. The interaction happens over common communication protocols which are transmitted over standard connectors.

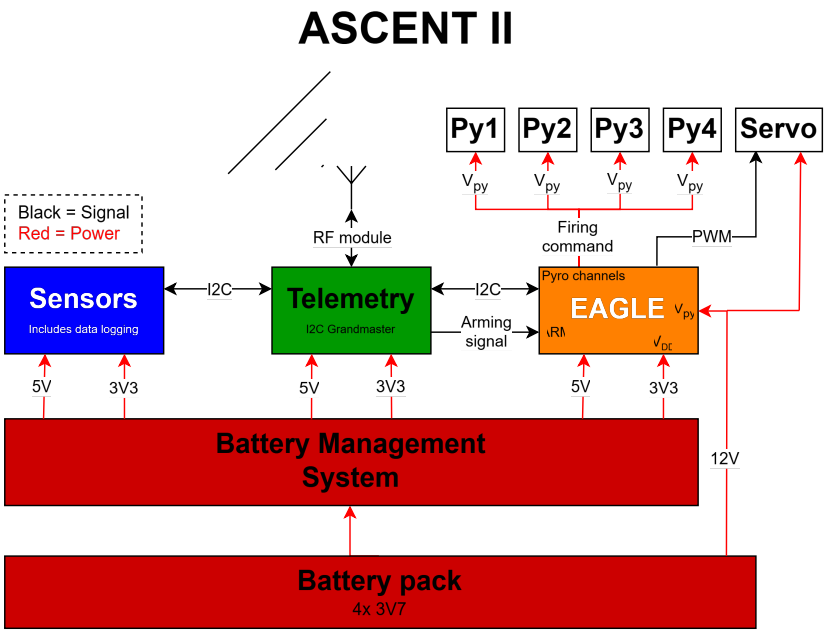


Figure 5.1.1: ASCENT II architecture

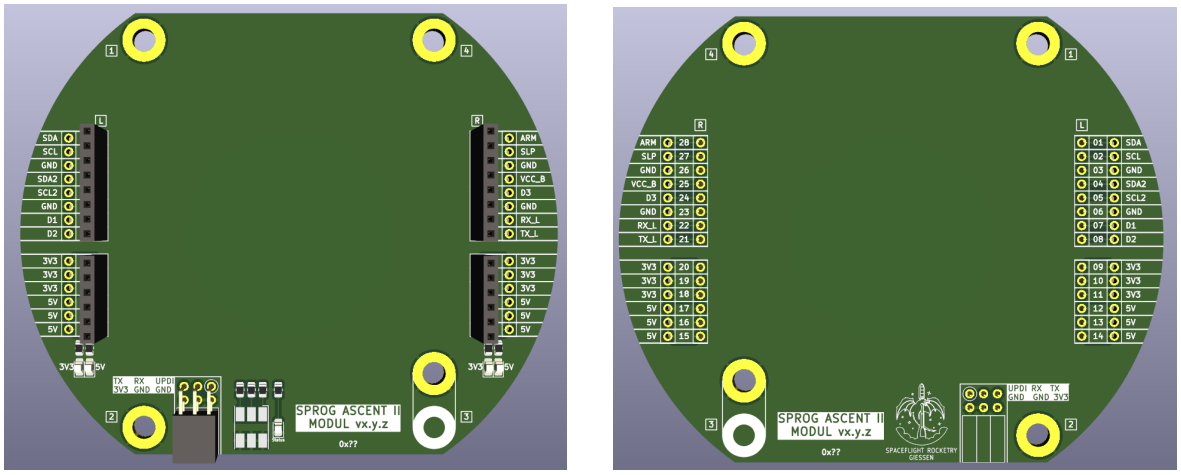
At the core of the "active" modules sits the telemetry module which serves as the grandmaster[1] in the I2C communication interface. This was chosen since it will be receiving and transmitting all necessary flight data to the ground station as well receive and manage all incoming commands from the ground. It initiates communication with the other modules when it is ready to receive data to be transmitted using so called "interrupts" on the I2C channel. This causes the slave module to interrupt its current software for a very short amount of time to answer the call from the telemetry module. Since the modules run asynchronously[2], there would otherwise be a chance of packet loss due to overbuffering. This could be caused by one module sending data quicker than the other communication partner can receive/read said data. Additionally, this also avoids the possibility of multiple modules trying to communicate over the same channel at once, which would result in the loss of all packets on the channel, due to the nature of I2C half-duplex architecture. One of the before mentioned commands is the arming signal and the deployment commands. As part of the requirements in chapter 4, it is necessary to provide more than one step in order to be able to deploy any kind of energetics on the launch pad. Without an active arming signal, which is given by a user-command[3], EAGLE is not able to fire any pyro-channel and therefore deploy any parachutes.
The sensor modules' primary objective is collecting and logging all the critical flight data that can be used for flight analysis after touchdown. It not only collects data about altitude and internal pressures, but also about the GNSS position and bending loads on the rocket using strain gauges. It also receives essential data about the state of flight from EAGLE using the UART communication protocol so that said data can also be logged. The logging happens on an SD-card as well as a flash chip. The latter being necessary for a hard touchdown as IC's are not prone to cracking under shock, whereas SD-cards can easily crack under shock loads.
The battery managements system is ASCENT II's only "passive" module, meaning that there is no software or microcontroller involved. It only utilizes passive elements such as DC-DC voltage converters, capacitors and resistors to execute its function. Its main objective is to down-regulate the incoming 14.8 V from the battery pack to 3.3 V and 5 V, which are then used by the other modules to power things such as microcontrollers and sensors (3.3 V) and such as the telemtry module (5 V). On top of that, it smooths out voltage spikes by using a combination of resistors and capacitors.
The last of ASCENT II's modules is EAGLE, which controls parachute deployment and airbrake actuations. It also features its own set of sensors in order to make it independent of the sensor modules. This was done, because the only way left to communicate between the two modules was UART (due to lack of I2C channels) or SPI. The latter being considered to complex to be implemented at the time and the other being considered significantly slower than I2C in terms of communication speed, which is not reliable enough for real-time decision making.
In total, ASCENT II consists of five PCB's (2 Sensor modules), which are all individually

[1]Each module has two I2C channels. "Grandmaster" refers to the master of the channel over which the modules communicate with each other, "master" refers to the channel which each module uses "locally" for communcation with e.g. sensors.
[2]They run at different loop timings and are not synchronized
[3]Has to be sent by a human-in-the-loop

designed along a certain design standard and PCB concept, which will be further discussed in subchapter 5.1.2.

5.1.2 PCB Concept

In order to ensure the modularized design and having a consistent designs across all modules, a PCB concept was implemented. The concept, as seen in figure 5.1.2, consists of some standard component that should included in every module. One of these components are the two pin connectors on the left and right of the PCB. These are the main connections between the modules and carry signals and voltage supply over common pins. Additionally, there is a programming interface on the bottom left of the top view, which can I either be connected to a commonly used USB to UART converter or a UPDI converter, which can both be used for programming the microcontroller, depending on which protocol it supports. To the right of said interface, there is two status LED's. One of them, is single color LED, whilst the other is a RGB LED. The single color LED can be used to show simple debugging steps, like that the microcontroller is running and executing its software. The RBG LED is supposed to display the status of the module. This could be something like green = running, red = error and other colors indicating what part of the software is being executed. These LED's main purpose is debugging and troubleshooting with visual feedback, which would be crucial during a launch campaign, where any kind of mistake should ideally be caught before final integration.
For integration purposes, the PCB also has standardized mounting holes, which can only be mounted in one specific way. That way, no PCB can be mounted incorrectly leading to the pins on the connector to be swapped around. When stacked, the PCB's are connected by the pin connectors and by threaded spacers which screw together through the mounting holes.
All of these standard features were either proven during or a learning of SPROG's inaugural launch.



(a) Top view

(b) Bottom view

Figure 5.1.2: PCB concept

5.1.3 EAGLE Architecture

EAGLE's architecture, which is shown in figure 5.1.3 slightly varies from that of the other modules. This comes down to the fact that EAGLE is designed to handle a broader variation of system i.e. airbrakes, parachute deployment and staging. To keep up reliability and minimize the effect of external influences, EAGLE does most of the flight computers tasks independently. This also includes its own set of sensors. The amount of sensors is based on the redundancy requirement in EuRoC's. For this reason, every type of sensor, except for the inertial measurement unit (IMU), has a redundant partner from an entirely different manufacturer. This is done so that no systematic issues of the sensor influence both readings, creating a more robust infrastructure. An IMU collects acceleration data (similar to the accelerometers) but also collects rotation speed. Since there are already two accelerometers, adding another acceleration reading will improve accuracy of the processed signal, but will not have a big impact of overall system reliability. The reason it was still added is purely for its capability to retrieve the rotation speed, which was added in case such a measurement might be needed in the future. In addition to sensors, it became apparent that additional input and output (IO) ports were needed. For this reason a (GP-)IO multiplexer was added, which added another 16 IO's to the already existing IO's of the microcontroller.
To communicate and process the data from the sensors and multiplexer, EAGLE features its own microcontroller, which uses its secondary I2C port to communicate with the sensors with itself as the master and each sensor as its slave. Furthermore, it communicates with the telemetry module using I2C where it acts as the slave of the grandmaster. If the arming signal was given by the telemetry module, it was possible for the main control unit (MCU) to send a voltage signal to the pyro-channels, which would lead to the firing current to be released. On top of that, the MCU can also send sensing voltages, which would not be high enough to ignite the pyrotechnics, to detect the current running through the system. If it was then able to detect any form of current, it was clear that a pyrotechnic igniter was installed and continuity on the channel was present. The MCU is also capable of sending PWM signals to a servo that would actuate the airbrakes.
During the development of ASCENT I, it was discovered that requiring a middle man to decode USB to UART signals in order to program the MCU, was time consuming and complex, because it required you to put both systems on the same power and ground reference, which made programming labour intense. To avoid this, with this iteration, EAGLE features a USB-C interface that includes all features of the USB to UART converter, which would let you read data from the MCU and program it with one interface, without connecting any additional wires.
There are also some external connections. These include the before mentioned I2C connection to the grandmaster, with EAGLE acting as its slave. It also includes the arming signal, without EAGLE is physically incapable of firing the pyro-channels. EAGLE also gets its main voltage supply V_{dd}, a 3.3 V connection and the supply voltage for the pyrotechnics V_{py} from external connections[4]. V_{dd} and V_{py} are then internally routed to the pyro-channels where they are used.

[4] Any external connection mentioned in this subchapter refers to connections coming from the pin connectors

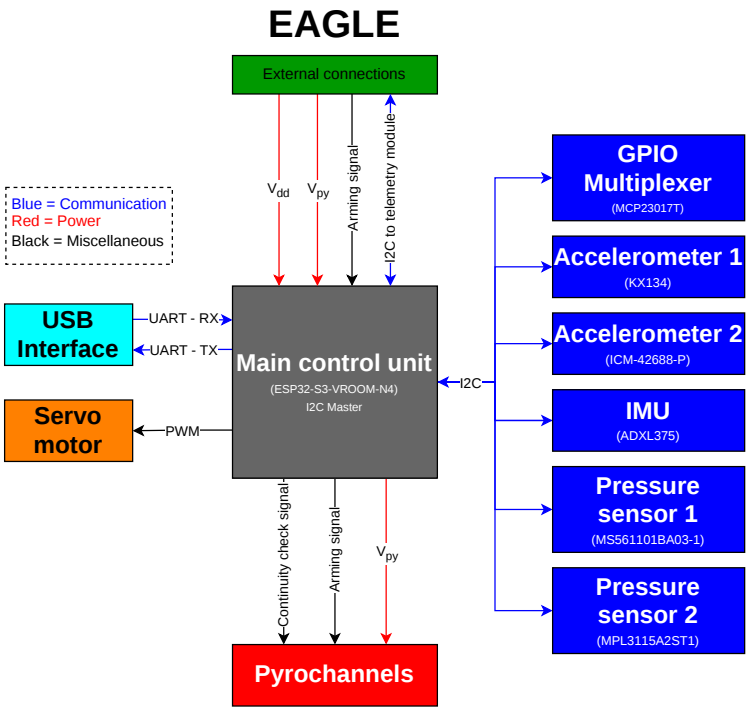


Figure 5.1.3: EAGLE architecture

5.2 Software Architecture

With EAGLE being in charge for many systems that would be considered saftey-critical, it is imperitive that the software is just as reliable as the hardware. Any unexpected behaviour can - in the worst-case scenario - cause a catastrophic failure, which could result in the loss of the rocket or damage to person and property. The overall architecture and decisions that led to this architecture will be highlighted in this subchapter.

5.2.1 State Machine

EAGLE's software is based on the concept of a state machine which was introduced in subchapter 2.2.1. The reason it was chosen as the architecture is due to the predictability and simplicity. On top of that, a rockets trajectory can be split into certain milestones or states (as discussed in subchapter 2.6). EAGLE's state machine consists of eight states, which are shown in figure 5.2.1. The states consist of "Pad idle" (0), "Initialized" (1), "Armed" (2), "Boost phase" (3), "Coasting" (4), "Apogee" (5), "Main deployment" (6), "Landed" (7) and "Abort" (8). Each state is assigned a number as part of an enumeration, which maps number values to strings for easier use in the software. The numbers of each state are represented in parentheses.

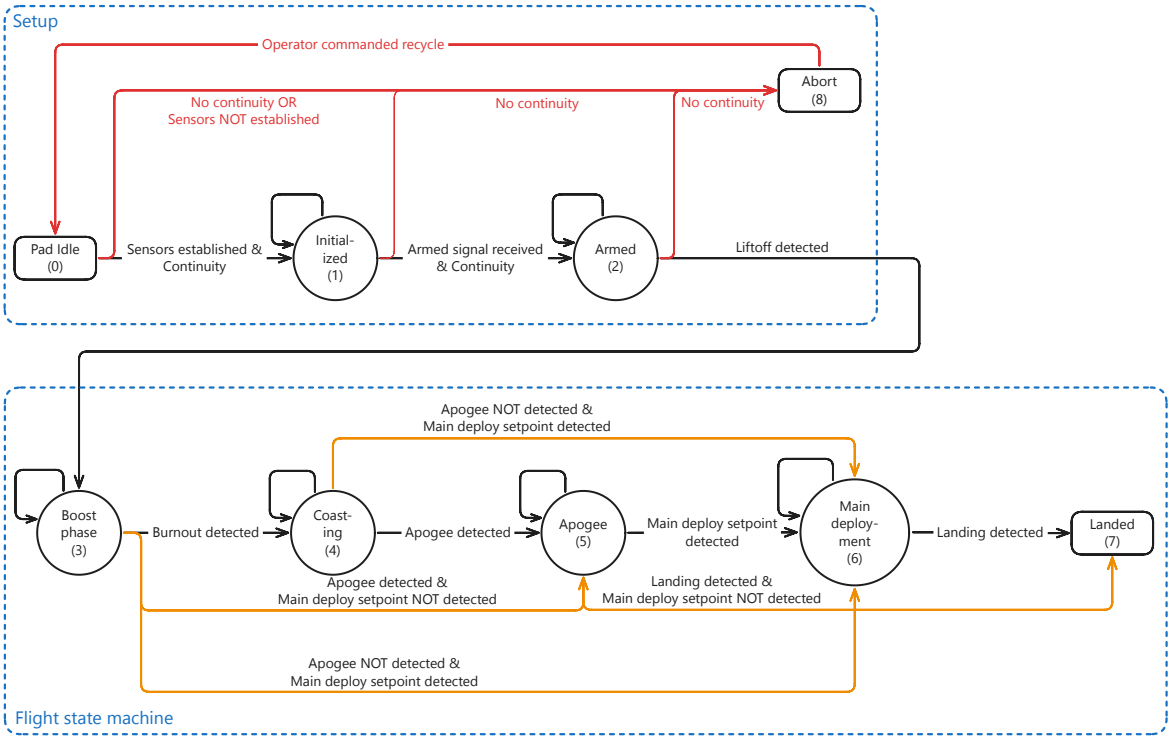


Figure 5.2.1: Software architecture

"Pad idle" is the starting state of the state machine. It is the default state that is assumed when EAGLE is powered on. At this stage, communication to the sensors is established by running an "I2C scanner", which sends a handshake (signal) to all 255 possible I2C addresses and waits for a response. If a response is received, communication to the sensor is considered established. Additionally, continuity of the pyro channels is checked. It detects a sensing current going through the pyrotechnic igniters. Detecting the sensing current on the minus port of the pyro channel implicates that the electrical circuit is closed, indicating a connected igniter. Each igniter can be tested individually by cycling through the individual pyro channels. At this point, anything that is off nominal would lead to an abort or recycle. Whilst EAGLE is made of many redundancies, it is unwise to launch with the knowledge that these redundancies have already been used. During this phase the vehicle is still unarmed and can quickly be taken down for inspection, which is why it makes sense to abort the launch should any abnormalities be found.

"Initialized" indicates that the microcontroller has successfully established communication with all sensors and peripherals, whilst also confirming that igniters are connected. In this state, the continuity to the igniters is still continuously checked, while waiting for the armed command from the telemetry via I2C.

Once the arming has been confirmed EAGLE enters the "Armed" state. In this state only continuity and liftoff detection are being checked for. Similarly to the prior states, any abnormalities on the continuity would result in an abort. Liftoff is detected by using the accelerometer data to detect a spike in acceleration, which would only happen when the booster is producing its main thrust. After liftoff, continuity will no longer be checked as the main objective is no longer to catch abnormalities, but to perform the main flight logic.

As already discussed in subchapter 2.6, a key characteristic of booster burnout is an accelera-

tion of $\approx 9.81 \frac{\mathrm{m}}{\mathrm{s}^2}$ on the axis perpendicular to the ground. When this is detected, the software enters the coasting phase. During this phase, staging will be initiated (if the rocket is two staged) and the airbrake logic will be executed, because it is the only phase in the ascent, where only gravitation and aerodynamics dictate the flight.
The "Apogee" state runs the logic required to deploy the drogue parachute. This is dictated by the EuRoC rules as shown in subchapter 3.2.2. At apogee, pyro channel 1 would be set to high igniting the first set of pyro charges. Additionally, there is logic which detects drogue deployment. If drogue deployment is not detected after a certain time, pyro channel 2 will be set to high, to ignite the secondary, redundant pyro charge for the drogue. Even without confirmed deployment, it constantly checks the current altitude versus the setpoint set for main parachute deployment.
"Main deployment" is essentially the same as apogee, but it used pyro channel 3 as primary and channel 4 as redundant igniter. Whilst this state follows the pyro state, it is not necessary to have a confirmed drogue deployment in order to try to deploy the main parachute. This is due to the fact, that a false negative on the confirmation is possible and it should not prevent the main parachute from deploying. On top of that, there is no downside to trying to deploy main parachute. Although there are low chances that the main would deploy without the drogue, it is not impossible, so it is reasonable to attempt the deployment.
As a last step, the vehicle enters the "Landed" state, which tells the telemetry to un-arm the pyro channels, so the rocket can safely be approached. It also tells the telemetry to initiate recovery procedures such as activating a beeper and power down systems to allow for maximum sending time of the touchdown location.
If there are any off nominal reports during the setup process, the "Abort" state may be entered, which tells the operator on the ground to hold the launch. However, this state does not have the capabilities to prevent the launch other than advice the operator to stand down. From an abort state, the telemetry can be used to make a decision whether to recycle the attempt, which means that the state machine would remotely be set back into the "Pad Idle" state by an operator or whether to abort the launch attempt and take the rocket off of the launch rail.

5.2.2 State Transitions

Each transition between states is governed by one or more boolean trigger conditions derived from sensor readings and system checks. Table 12.2.1 and table 12.2.2 specify these conditions for the setup phase and flight phase, respectively. Within these tables, a dash ("–") denotes a "do not-care condition": the corresponding flag has no influence on whether the transition occurs, and may hold either value.
Following power-up, the system resides in the initial setup/pad idle state (0). If both the sensors established and continuity flags evaluate to TRUE, initialization is considered complete and the system transitions to the initialized state (1), independent of the armed flag's state at that point. Conversely, if both flags are FALSE, setup is deemed to have failed and the system transitions directly to the abort state (8).
From state 1, the system remains unarmed as long as continuity is maintained and the armed flag is FALSE (transition $1 \rightarrow 1$). If the armed flag becomes TRUE while continuity is still

present, the system transitions to the armed state (2). Should continuity be lost at any point while in state 1, the system transitions to the abort state (8), regardless of the other flags.
State 2 behaves analogously with respect to continuity: the system remains in state 2 as long as continuity is present ($2 \rightarrow 2$), and transitions to the abort state (8) if continuity is lost.
Once setup completes successfully, the system enters the armed state (2), from which it transitions to the boost state (3) upon detection of liftoff. (As shown in table 12.2.2, state 2 additionally reverts to the abort state (8) if continuity is lost prior to liftoff). If the system enters an abort state, an operator on the ground may decide to abort the launch attempt or to recycle the state machine, forcing it back into the pad idle state, from where operations would start over again.
After detecting liftoff, the system monitors for booster burnout, apogee and main deployment. Each variable can lead to a different transition, as it is possible that either one of these steps is missed. However, it is important that EAGLE remains capable of deploying parachutes even if one or more events have been missed. For that reason, detecting apogee or main deployment altitude, would result in the state transition into the apogee ($3 \rightarrow 5$) or main deployment ($3 \rightarrow 6$) state. If the event monitoring is nominal, the coasting phase ($3 \rightarrow 4$) would be entered. Im the off-nominal case, the coasting phase would be skipped and all airbrake actuation would not be executed. This is done to ensure EAGLE's main objective, which is the successful and safe recovery of the rocket.
Analogous to the boost state, the system can jump into multiple outcomes, in this case into apogee ($4 \rightarrow 5$) and into main deployment ($4 \rightarrow 6$). This is done for the same reason of being able to control the parachutes even if an event has failed to get noticed.
When in the apogee state, the state machine has the nominal case of detecting main deployment ($5 \rightarrow 6$). However, there is still a chance that such an event would be missed, in which case the rocket would fall to the ground under drogue parachute, which would cause a relatively high impact speed. In this case, the rocket still needs to be able to inert (unarm) itself, if it survived the impact and therefore needs to be able to detect the landing, which would cause a transition into the landed state ($5 \rightarrow 7$).
If the vehicle does deploy the main parachute or at least enters state 6, it can only transition into landed as it is the last step in the flight sequence of events.
The events "apogee detected", "main parachute altitude detected" and "landing detected" can each be redundantly triggered by a ground operator command or by timeout. When EAGLE detects liftoff it starts timers which evaluate if an event could have been reached based on timings from simulations. So even if EAGLE was to not detect apogee or main deployment and still managed to survive the impact, a timer would cause the rocket to inert itself. Similarly, this serves as backup for missed apogee detecting by the sensor values.

5.2.3 PID Controllers

TBD

5.2.4 High-Level Code Architecture

Similarly to the state machine, the software is split into a main function and a setup code. The key difference is that the setup code is only called once, whereas the main function is a loop as seen in the code example 5.1 . This is part of the arduino framework which was chosen for this project because of its wide range of open-source libraries, which could be used for reading sensors or communicating via UART or I2C.

```
1    void setup{
2        //runs setup state machine
3        if(liftoff detected && continuity detected)
4        state = boost; //start main loop
5    }
6    int main {
7        while(state != landed){
8            //runs flight state machine
9            if(execution time > set execution time)
10           skip.to.next.iteration;
11       }
12   }
```

Listing 5.1: Top level code design

Because of the time critical nature of the system, it is important that the software runs deterministic i.e. that it runs its main loop in a pre-defined amount of time. This is important, to avoid unected behavior. This can be enforced by e.g. using interrupts, which completely stops the loop iteration if this set amount of time has been exceeded. The time for the loop to finish is set by creating a statistic about how long the loop takes to run once over multiple iterations, but can also be pre-defined as a requirement. In this case, it was decided to base it on the average execution time, because whilst it is important to have a deterministic code, parachute deployment may happen within the vicinity of apogee and the main deployment altitude as discussed in subchapter 4.3.

5.2.5 Main Loop Architecture

The main loop, which continuously runs ones the setup has been completed, is the core logic of the entire code base. To stay with the readability, maintainability and modularized concept of the state machine, the main loop is split into its core functionalities: Reading, conversion, processing, state machine and writing outputs (as shown in figure 5.2.2).

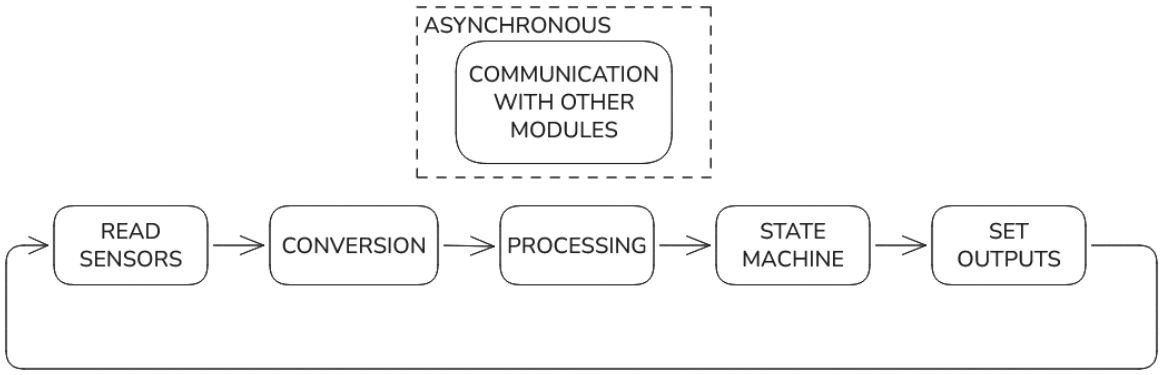


Figure 5.2.2: Software architecture

By splitting the functionality, the software becomes more scalable, because to add functionality, only small parts of the code have to be changed, which are at a defined position as opposed to being one continuous loop or function.
As the name suggests, the "Read sensors" part is meant to read any IO's available. This could be something as trivial as a digital input or a sensor. At this point, only data directly provided by the sensor is available.
In the conversion step that data is converted from binary values from the sensors into physical units such as bar, m or m/s^2. It also converts the physical units into values that are not directly provided by sensors such as integrated values or converted values like the conversion from bar to m using the barometric formula.
The processing does real-time data processing using common concepts like a moving average or a kalman filter. This increases the accuracy of the data, which in turn makes the detection of milestones more reliable. Additionally, the converted and processed data is then used to set flags such as "apogee reached" which will be used in the state machine to assess the state and decide how to continue.
The state machine uses the processed data and evaluate its state. Based on the state it is in, it would trigger certain actions, such as payload deployment. The state machine defines what outputs will need to be set based on the state transition and trigger additional logic, if necessary.
Finally, the outputs, which have been defined in the state machine are written to the actual hardware i.e. the output pins or buses. This is done as a last step so that there are no race-conditions created, which would cause two different states being set to the same output.
As shown in figure 5.2.2, communication with other ASCENT modules happens asynchronously. This means that the communication does not happen cyclically within the loop and outside of the loops time frame. This is done by using a software interrupt, which is triggered by the telemetry module. This is done by assigning highest scheduler priority to the communication task when the telemetry module queries/requests communication. This immediately stops the ongoing tasks until communication is done. Optionally, this could also be done as a parallel task on another CPU core, if applicable, so the main loop is never interrupted.

6 Preliminary Design Phase

Based on prior experience at SPROG, it was decided that each system would need to go through a preliminary design phase during which the requirement and architecture that were defined were to be tested for feasibility. This is done, to ensure that no resources are lost to small mistakes, which were not detected, because of a lack of knowledge or experience. This was an issue that poised SPROG's early stage developments. During the development of ASCENT I, PCB's were designed and ordered early on with minor to major design flaws costing the association time and resources, which were limited during that time due to the young age of the group. This design phase is mostly about understanding and selecting the correct sensors and confirm the design on basic open questions. For example, the sensor module phased the question whether to use a built in antenna for the GNSS module or an external antenna. The only relevant open question during this phase was the selection of sensors. The process that were taken to limit the amount of unknowns in the PCB design phase, will be illuminated in this chapter.

6.1 Component Selection

The first step in the preliminary design phase is the selection of components. The requirements dictate the amount of sensors and overall architecture but not which components to use exactly. Therefore, the first major step is to decide on the sensors that were to be used. However, there is no final decisions at this point. This is because each selected sensor had to go through initial testing on breakout boards and had to match certain expectations, which were evaluated during this process.

The requirements and the consequent architecture required five sensors in total: Two accelerometers, two pressure sensor and one inertial measurement unit (IMU) to detect all data necessary for EAGLE's operation. To have true redundancy, it makes sense to not choose two sensors of the same kind from the same manufacturer or even the same product family. By doing so, the risk of having two sensors drop out because of a similar systematic issue becomes less likely.

Some characteristics that are taken into account are things such as the supply voltage V_{DD}, power consumption, noise, accuracy/resolution and measurement range. Additionally, it has to be taken into account that I2C uses unique addresses, with each peripheral being able to have two assigned addresses, the primary and backup address, should either be taken by another peripheral. Therefore, it needs to be ensured that no I2C peripheral has overlapping addresses. Furthermore, the existence and ease of use of a sensor library, was taken into account. By doing so, a sensor could essentially become a plug-and-play system for testing and software related issues were minimized as some of these software libraries have existed and been used for multiple years.

6.2 Microcontroller

The microcontroller is the heart of all operations and needs to be the most reliable component on the PCB. It collects all the data from the sensors and distributes it to the telemetry system and data storage as well as fire pyro-channels and actuate the servos and be able to read analogue signals. Therefore, the selected microcontroller needs a wide range of pin-types.

6.2.1 AVR128DB64

The AVR128DB64 was the first choice solely based on past experience and ease of programming. It has an easy to understand pinout and 64 pins and runs on the arduino framework. The AVR128DB28 (same family of controllers) has been used in a previous telemetry system meaning there was a lot of experience using this type of microcontroller. Additionally, it allows communication via SPI, UART and I2C. The latter being used for communication with all sensors and between the communication with the telemetry subsystem. It also featured a relatively new programming protocol called "Unified Program and Debug Interface" (UPDI), which only requires one pin for programming instead of two dedicated UART ports like other microcontrollers, making programming substantially easier.
In hindsight, there should have been less emphasize on past experiences as they allowed for an oversight on the computing power of the chip, which is arguable the most important characteristic. After connecting and testing multiple sensors to the microcontroller, it was discovered the RAM overflowed with data during the first loop, rendering the microcontroller useless for usage in EAGLE.

6.2.2 ESP32-S3-Vroom-N4-1u

After running into issues with the AVR128DB64 a team member of the electronics working group within the association suggested to test the ESP32 as it is a dual-core microcontroller and has more RAM available. It also features all common communication protocols, but needs to be programmed using UART, rendering two pins unusable for any other purposes. The programming also needs to additional pins called "EN" and "IO0" which need to be pulled to GND in a specific sequence to put the microcontroller into the so called "programming mode". However, it did feature a D+ and D- pin meaning a USB interface can easily be connected for programming and data-reading. In addition to that, it is an IC meaning that it has additional features such as an built-in WiFi module and antenna. To save space it was decided to use the "-1u" version, because that means that the ESP32 would come without an antenna but with a coax-connector. However, all of this came at the cost of the arduino framework as ESP32s run on their own firmware and framework. In some cases however, the ESP32 is capable of running arduino code, meaning that the code only needs to be adapted slightly.
For those reasons, the ESP32-S3-Vroom-N4-1u was chosen as the microcontroller for EAGLE.

6.3 Design Evaluation and Revision

7 Software Implementation

This chapter describes the discrete implementation of the software architecture using pseudo-code snippets. It is meant to highlight the challenges and design decisions made during the development of the software.

7.1 Internal Data Processing

EAGLE collects a lot of data, which it has to process every single loop cycle. For example, all altitude readings are derived from other physical units such as the barometric pressure and acceleration. However, the conversion from these values to useful data that can be processed an used by controllers and other parts of the software, it has to go through a series of conversion steps. These steps will be highlighted in this subchapter.

7.1.1 Integration of IMU data

7.2 State machine

The implementation of the state machine is done by using an enumeration, a list of string values that are assigned to numbers, for abstracting the states, which in turn makes the code more readable. In the main loop, a switch-case statement, essentially a large number of if-else-statements, is executed in every iteration (as shown in code example 7.2). The main loop only starts executing logic, once the setup process is complete, liftoff was detected and the state has been changed to "boost". The switch-case statement checks which state is currently active and only executes that specific state's logic.

```
1     enum States {
2         pad_idle = 0,
3         initialized = 1,
4         armed = 2,
5         boost = 3,
6         coast = 4,
7         apogee = 5,
8         main_deploy = 6,
9         landed = 8
10    };
```

Listing 7.1: States enumeration

```
1     while(state != landed){
2         getSensorData(); //retrievs sensor data
3         switch(state) {
```

```
            case boost:
            //execute burnout monitoring
            break;
            case coast:
            //execute airbrake and staging logic
            break;
            case apogee:
            //execute drogue deployment logic
            break;
            case main_deploy:
            //execute main deployment logic
            break;
            case landed:
            //execute post-touchdown/inerting logic
            break;
            default:
            //do nothing until a known state is set by the setup state
            //machine
        }
    }
```

Listing 7.2: Flight state machine

In each state, outputs are set and the state transition triggers are evaluated. If a transition trigger is detected, the global state variable "state" (can be changed and accessed by all functions) is changed and the respective case is executed. A check for a state transition trigger can look like the following:

```
    switch(state) {
        case boost:
        //boost state logic
        if(burnout detected)
        state = burnout;
        break;

```

Listing 7.3: State transition logic

Because of the break statement, the switch-case statement finishes for this iteration and will assume the burnout state in the next iteration.
In order to be able to process data within each case, sensor data has to be retrieved. This is done, before calling the switch-case statement, because every state needs the sensor data and it therefore makes the code more readable. In this sensor reading function, all 5 sensors are called and read. Afterwards, the data gotten from these sensors is processed using common data filters to narrow down the error in data calculations. On top of that, multiple sources of the same data will be calculated into one single data point, so that the case only has to call the processed data and does not have to do any advanced arithmetics.

8 PCB Design - ASCENT II

8.1 Power Budget

9 Verification and Testing

10 Discussion

11 References

[1] Microchip Technology Inc. *AN5603: Power Supply Decoupling and Layout Considerations*. 2024. URL: https://www.microchip.com/en-us/application-notes/an5603 (visited on 10/28/2026).

[2] Texas Instruments. *AN-1261: Power Supply Effects on Noise Performance*. 2015. URL: https://www.ti.com/lit/an/snaa022c/snaa022c.pdf (visited on 10/28/2026).

[3] UNIKEYIC ELECTRONICS. *Fundamentals of Passive Filters and Passive Low-Pass Filters in Circuit Design*. 2025. URL: https://www.unikeyic.com/blog/passive-filters-and-passive-low-pass-filters.html (visited on 10/28/2026).

[4] C. K. Alexander and M. N. O. Sadiku. *Fundamentals of Electric Circuits*. 7th ed. McGraw-Hill, 2021.

[5] Texas Instruments. *Understanding Schmitt Triggers*. 2025. URL: https://www.ti.com/document-viewer/lit/html/scea046 (visited on 10/28/2026).

[6] Ramesh Sathiyamoorthy. *I2C Bus*. 2015. URL: https://www.mikrocontroller.net/attachment/372171/I2C_bus.pdf (visited on 10/21/2026).

[7] Rhode & Schwarz. *Understanding UART*. N/A. URL: https://www.rohde-schwarz.com/uk/products/test-and-measurement/essentials-test-equipment/rs-essentials-digital-oscilloscopes/understanding-uart_254524.html?change_c=GB (visited on 10/27/2026).

[8] Art Pini. *Why and How to Use the Serial Peripheral Interface to Simplify Connections Between Multiple Devices*. 2019. URL: https://www.digikey.de/en/articles/why-how-to-use-serial-peripheral-interface-simplify-connections-between-multiple-devices?srsltid=AYCX9vKBxDtWGIJ43NI93qSCq9rTY4minwFCzgOA2ra_FM8T_IulzrZI (visited on 10/27/2026).

[9] Dr. Rainer Kirchhartz, Marcus Hörschgen-Eggers, and Wolfgang Jung. "Sounding Rockets are unique Experimental Platforms". In: Sept. 2018. Chap. Typical Applications of Sounding Rockets in Europe, p. 3. URL: https://elib.dlr.de/125360/.

[10] Pepermans et al. "Systematic Design of a Parachute Recovery System for the Stratos III Student Built Sounding Rocket". In: June 2018. Chap. Detailed Design, pp. 7–11. URL: https://www.researchgate.net/publication/325964775_Systematic_Design_of_a_Parachute_Recovery_System_for_the_Stratos_III_Student_Built_Sounding_Rocket.

[11] Jason Gläser. "Mechanical Design of Electronic Actuation & Guidance for Landing and Elevation (EAGLE) system". Student project. University of applied sciences Giessen, 2026. Chap. Flight trajectory, p. 3.

[12] Michael Protogerakis. *Druckkräftegleichgewicht am infinitesimalen Volumenelement in y-Richtung*. 2005. URL: https://commons.wikimedia.org/wiki/File:P_gl_gew_y.gif.

[13] International Civil Aviation Organization. *Manual of the ICAO standard atmosphere*. Doc.: 7488. Secretary General of International Civil Aviation Organization, 1993.

[14] Deutsches Zentrum für Luft- und Raumfahrt. *MORABA – Suborbital Space Flight with Research Rockets*. 2026. URL: https://moraba.de/en/moraba/ (visited on 09/28/2026).

[15] Prof. Chiara Manfletti. *Spectacular Success for TUM Spin-off Isar Aerospace*. 2026. URL: https://www.tum.de/en/news-and-events/all-news/press-releases/details/spectacular-success-for-tum-spin-off-isar-aerospace (visited on 09/29/2026).

[16] Bundesverband Studentischer Raumfahrt e.V. *Raumfahrt. Studentisch. Gestalten.* 2026. URL: https://bvsr.space/ (visited on 09/29/2026).

[17] Inês d'Ávila. *System Requirement Document*. Ref.: PTS_EDU_EuRoC_ST_001928. Portuguese Space Agency, 2025.

12 Appendix

12.1 Artificial Intelligence

Some (sub-)chapters have been rewritten, reworked or corrected using generative AI tools such as ChatGPT and Claude. The used models and affected chapters will be noted in this Appendix chapter. The below mentioned models have only been used for writing and researching. No AI agents have been used in the design of EAGLE or its software architecture.

Table 12.1.1: AI usage within this thesis

Model	**Affected chapter**	**Type of use**
Claude Sonnet 5.5	Subchapter 3.1 (MORABA) and Introduction text of chapter 3.2 (European Student Rocketry)	Text generation and optimization
Claude Sonnet 5.5	Every chapter	Proofreading and correction of grammatical and spelling mistakes
GPT-5.6 Luna	Every chapter	Proofreading and correction of grammatical and spelling mistakes

12.2 Tables

12.2.1 State transition truth tables

Table 12.2.1: EAGLE setup state transitions

Transition description		**Transition trigger**			
Transition	**Cause**	**Sensors established**	**Continuity**	**Armed**	**Recycle**
$0 \rightarrow 1$	Initialization complete	TRUE	TRUE	-[1]	FALSE
$0 \rightarrow 8$	Setup failed	FALSE	FALSE	-	FALSE
$1 \rightarrow 1$	No change	-	TRUE	FALSE	FALSE
$1 \rightarrow 2$	Armed signal	-	TRUE	TRUE	FALSE
$1 \rightarrow 8$	No continuity	-	FALSE	-	FALSE
$2 \rightarrow 2$	No change	-	TRUE	FALSE	FALSE
$2 \rightarrow 8$	No continuity	FALSE	-	-	FALSE
$8 \rightarrow 0$	Recycle command	-	-	-	TRUE

[1] "-" means that the state of this flag is unimportant for the transition trigger

Table 12.2.2: EAGLE flight state transitions

Transition description		**Transition trigger**				
Transition	**Cause**	**Liftoff detected**	**Burnout detected**	**Apogee detected**	**Main deploy setpoint detected**	**Landing detected**
$2 \rightarrow 3$	Liftoff detected	TRUE	-	-	-	-
$3 \rightarrow 3$	No change	-	FALSE	-	-	-
$3 \rightarrow 4$	Burnout detected	-	TRUE	-	-	-
$3 \rightarrow 5$	Apogee detected	-	-	TRUE	FALSE	-
$3 \rightarrow 6$	Main deploy setpoint detected	-	-	FALSE	TRUE	-
$4 \rightarrow 4$	No change	-	-	FALSE	FALSE	-
$4 \rightarrow 5$	Apogee detected	-	-	TRUE	-	-
$4 \rightarrow 6$	Main deploy setpoint detected	-	-	FALSE	TRUE	-
$5 \rightarrow 5$	No change	-	-	-	FALSE	-
$5 \rightarrow 6$	Main deploy setpoint detected	-	-	-	TRUE	-
$6 \rightarrow 6$	No change	-	-	-	-	FALSE
$6 \rightarrow 7$	Landing detected	-	-	-	-	TRUE

12.2.2 Sensor characteristics

Table 12.2.3: Inertial Measurement Unit(IMU) characteristics Nr.1

	BMI160					**MPU6500**				
AccelerometerSpecs										
Measurement Range	2 g	4 g	8 g	16 g		2 g	4 g	8 g	16 g	
Output Resolution	16 bits					16 bits				
Sensitivity (Typical)	16384 $\frac{\text{LSB}}{\text{g}}$	8912 $\frac{\text{LSB}}{\text{g}}$	4096 $\frac{\text{LSB}}{\text{g}}$	2048 $\frac{\text{LSB}}{\text{g}}$		16384 $\frac{\text{LSB}}{\text{g}}$	8192 $\frac{\text{LSB}}{\text{g}}$	4096 $\frac{\text{LSB}}{\text{g}}$	2048 $\frac{\text{LSB}}{\text{g}}$	
Sensitivity change due to temperature	**x**	**y**		**z**		**x**	**y**		**z**	
	$\pm 0.03 \frac{\%}{^\circ\text{C}}$					$\pm 0.026 \frac{\%}{^\circ\text{C}}$				
0g Offset	± 0.04 g					± 0.06 g				
Noise	± 1.8 mg-rms					Low Noise Mode: $\pm 300 \frac{\mu\text{g}}{\sqrt{\textbf{Hz}}}$				
Operating Voltage V_s/V_{DD}	**Min**	**Typical**			**Max**	**Min**	**Typical**			**Max**
	1.71 V	3.0 V			3.6 V	1.71 V	1.8 V			3.45 V
Interface Voltage $V_{\textbf{DDIO}}$	1.2 V	2.4 V			3.6 V	1.71 V	1.8 V			3.45 V
Output Data Rate (ODR)	**Min**		**Max**			**Min**		**Max**		
	12.5 Hz		1600 Hz			4 Hz		4000 Hz		
Current Consumption	**Accelerometer**		**Gyroscope**	**Both**(On)	**None**(Stby)	**Accelerometer**	**Gyroscope**	**Both**(On)	**None**(Stby)	
	300 µA		900 µA	990,µ	10 µA	0.45 mA	3.2 mA	3.4 mA	1.6 mA	
Power Consumption	**For Typical** $V_{\textbf{DD}} = 3.0$ **V**					**For Typical** $V_{\textbf{DD}} = 1.88$ **V**				
	0.90 mW		2.70 mW	2.97 mW	0.03 mW	0.81 mW	5.76 mW	6.12 mW	2.88 mW	
Temperature Range	-40 ... +85 °C					-40 ... +85 °C				
GyroSpecs										
Measurement Range	125 $\frac{\text{deg}}{\text{s}}$	250 $\frac{\text{deg}}{\text{s}}$	500 $\frac{\text{deg}}{\text{s}}$	1000 $\frac{\text{deg}}{\text{s}}$	2000 $\frac{\text{deg}}{\text{s}}$	250 $\frac{\text{deg}}{\text{s}}$	500 $\frac{\text{deg}}{\text{s}}$	1000 $\frac{\text{deg}}{\text{s}}$	2000 $\frac{\text{deg}}{\text{s}}$	
Sensitivity	262.4 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	131.2 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	65.6 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	32.8 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	16.4 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	131 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	65.5 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	32.4 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	16.4 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	
Sensitivity Change Due to temperature	$\pm 0.02 \frac{\%}{\text{C}}$					$\pm 3\,\%$[1]				
Zero-rate Offset	$\pm 3 \frac{\text{deg}}{\text{s}}$					$\pm 5 \frac{\text{deg}}{\text{s}}$				
Noise	$0.07 \frac{\text{deg}}{\text{s}}$-rms					$\pm 0.01 \frac{\text{deg}}{\text{s}}$-rms				

Table 12.2.4: Inertial Measurement Unit(IMU) characteristics Nr.2

	BMI160								**MPU6500**							
AccelerometerSpecs																
Measurement Range	2 g		4 g		8 g		16 g		2 g		4 g		8 g		16 g	32 g
Output Resolution	16 bits								16 bits							
Sensitivity (Typical)	16384 $\frac{\text{LSB}}{\text{g}}$		8912 $\frac{\text{LSB}}{\text{g}}$		4096 $\frac{\text{LSB}}{\text{g}}$		2048 $\frac{\text{LSB}}{\text{g}}$		16384 $\frac{\text{LSB}}{\text{g}}$		8192 $\frac{\text{LSB}}{\text{g}}$		4096 $\frac{\text{LSB}}{\text{g}}$		2048 $\frac{\text{LSB}}{\text{g}}$	1024 g
Sensitivity change due to temperature	$\pm 0.005 \frac{\%}{^\circ\text{C}}$								$\pm 0.01 \frac{\%}{^\circ\text{C}}$							
0g Offset	±20 mg								±10 mg							
Noise	**x and y-axis**				**z-axis**				**All axis**							
	0.65 mg-rms				0.70 mg-rms				0.70 mg-rms					0.80 mg-rms		1.1 mg-rms
Operating Voltage V_s/V_{DD}	**Min**		**Typical**				**Max**		**Min**		**Typical**			**Max**		
	1.71 V		1,8 V				3.6 V		1.71 V		1.8 V			3.6 V		
Interface Voltage $V_{\mathbf{DDIO}}$	1.71 V		1,8 V				3.6 V		1.08 V		1.8 V			3.6 V		
Output Data Rate (ODR)	**Min**				**Max**				**Low Noise Mode** (LNM)				**Low Power Mode** (LPM)			
	1.5625 Hz				32000 Hz				**Min:** 12.5 Hz		**Max:** 6400 Hz		**Min:** 1.5625 Hz		**Max:** 400 Hz	
Current Consumption	**3-Axis Accel.**		**3-Axis Gyro**		**6-Axis Accel. + Gyro**		**Sleep Mode**		**6-Axis Accel. + Gyro**[2]		**3-Axis Accel.**	**3-Axis Gyro**	**6-Axis Accel. + Gyro**[3]		**3-Axis Accel.**	**3-Axis Gyro**	**Sleep Mode**
	0.28 mA		0.73 mA		0.88 mA		0.75 µA		420 µA		120 µA	360 µA	220 µA		67 µA	210 µA	2.2 µA
Power Consumption	**For** $V_{\mathbf{DD}} = 3.3$ **V**								**For** $V_{\mathbf{DD}} = 3.3$ **V**							
	0.924 mW		2.409 mW		2.904 mW		24.75 µW		1,386 mW		396 µW	1.118 mW	726 µW		221.1 µW	693 µW	7.26 µW
Temperature Range	-40 ... +85 °C								-40 ... +85 °C							
GyroSpecs																
Measurement Range	15.625 $\frac{\text{deg}}{\text{s}}$	31.25 $\frac{\text{deg}}{\text{s}}$	62.5 $\frac{\text{deg}}{\text{s}}$	125 $\frac{\text{deg}}{\text{s}}$	250 $\frac{\text{deg}}{\text{s}}$	500 $\frac{\text{deg}}{\text{s}}$	1000 $\frac{\text{deg}}{\text{s}}$	2000 $\frac{\text{deg}}{\text{s}}$	15.625 $\frac{\text{deg}}{\text{s}}$	31.25 $\frac{\text{deg}}{\text{s}}$	62.5 $\frac{\text{deg}}{\text{s}}$	125 $\frac{\text{deg}}{\text{s}}$	250 $\frac{\text{deg}}{\text{s}}$	500 $\frac{\text{deg}}{\text{s}}$	1000 $\frac{\text{deg}}{\text{s}}$	2000 $\frac{\text{deg}}{\text{s}}$	4000 $\frac{\text{deg}}{\text{s}}$
Sensitivity	2089.2 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	1048.6 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	524.3 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	262 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	131 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	65.5 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	32.8 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	16.4 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	2089.2 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	1048.6 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	524.3 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	262 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	131 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	65.5 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	32.8 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	16.4 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$	8.2 $\frac{\text{LSB}}{\frac{\text{deg}}{\text{s}}}$
Sensitivity Change Due to temperature	$\pm 0.005 \frac{\%}{^\circ\text{C}}$								$\pm 0.01 \frac{\%}{^\circ\text{C}}$								
Zero-rate Offset	$\pm 0.5 \frac{\text{deg}}{\text{s}}$								$\pm 0.3 \frac{\text{deg}}{\text{s}}$								
Noise	$0.028 \frac{\text{deg}}{\text{s}}$-rms								$\pm 0.038 \frac{\text{deg}}{\text{s}}$-rms								

[1] maximum absolute offset
[2] At 50 Hz ODR
[3] At 200 Hz ODR

Table 12.2.5: Accelerometer characteristics

	ADXL345				**ADXL343**				**KX134**			
Measurement Range	2 g[1]	4 g	8 g	16 g	2 g	4 g	8 g	16 g	8 g	16 g	32 g	64 g
Output Resolution	10 bits	11 bits	12 bits	13 bits	10 bits	11 bits	12 bits	13 bits	16 bits			
Sensitivity (Typical)	256 $\frac{\text{LSB}}{\text{g}}$	128 $\frac{\text{LSB}}{\text{g}}$	64 $\frac{\text{LSB}}{\text{g}}$	32 $\frac{\text{LSB}}{\text{g}}$	256 $\frac{\text{LSB}}{\text{g}}$	128 $\frac{\text{LSB}}{\text{g}}$	64 $\frac{\text{LSB}}{\text{g}}$	32 $\frac{\text{LSB}}{\text{g}}$	4096 $\frac{\text{LSB}}{\text{g}}$	2048 $\frac{\text{LSB}}{\text{g}}$	1024 $\frac{\text{LSB}}{\text{g}}$	512 $\frac{\text{LSB}}{\text{g}}$
Sensitivity change due to temperature	**x**	**y**	**z**		**x**	**y**	**z**		**x**	**y**	**z**	
	±0.01 $\frac{\%}{°\text{C}}$				±0.01 $\frac{\%}{°\text{C}}$				±0.01 $\frac{\%}{°\text{C}}$		±0.03 $\frac{\%}{°\text{C}}$	
0g Offset	±0.15 g		±0.25 g		±0.35 g				±0.025 g			
Noise	±0.75 LBS-rms		±1.10 LBS-rms		±1.10 LBS-rms				±0.70 LBS-rms			
Operating Voltage V_s/V_DD	**Min**	**Typical**		**Max**	**Min**	**Typical**		**Max**	**Min**	**Typical**		**Max**
	2.0 V	2.5 V		3.6 V	2.0 V	2.5 V		3.6 V	1.7 V	2.5 V		3.6 V
Interface Voltage $V_\textbf{DDIO}$	1.7 V	1.8 V		V_DD	1.7 V	1.8 V		V_DD	1.2 V	-		3.6 V
Output Data Rate (ODR)	**Min**	**Max**			**Min**	**Max**			**Min**	**Typical**		**Max**
	0.1 Hz	3200 Hz			0.1 Hz	3200 Hz			0.781 Hz	50 Hz		25600 Hz
Current Consumption	**ODR≥100 Hz**	**ODR<10 Hz**	**Standby**		**ODR≥100 Hz**	**ODR<10 Hz**	**Standby**		**Performance**	**Low Power**	**Standby**	
	100 µA	30 µA	0.1 µ		100 µA	30 µA	0.1 µA		148 µA	0.53 µA	0.5 µA	
Power Consumption	**Min**	**Typical**	**Max**		**Min**	**Typical**	**Max**		**For $V_\textbf{DD}$ = 3.3 V**			
	0.28 mW	0.35 mW	0.5 mW		0.28 mW	0.35 mW	0.5 mW		≈0.49 mW	1.79 µW	1.65 µW	
Temperature Range	-40 ... +85 °C				-40 ... +85 °C				-40 ... +105 °C			
Weight	30 mg				30 mg				-			

[1] g referring to gravitational acceleration of 9.81 $\frac{\text{m}}{\text{s}^2}$

Table 12.2.6: Pressure sensor characteristics

	HSCMAND015PASA5			**MS561101BA03-50**			**MPL3115A2ST1**[a]			**MS560702BA03-50**		
Pressuresensor Specs												
Measurement Range	1.6 **mbara** − 1.0 **bara**	160 **Pa** − 1 **MPa**	0.5 **psi** − 14.5 **psi**	450 **mbara** − 1.1 **bara**	45 **kPa** − 110 **kPa**[b]	6.5 **psi** − 16.0 **psi**	500 **mbara** − 1.1 **bara**	50 **kPa** − 110 **kPa**[c]	7.3 **psi** − 16.0 **psi**	300 **mbara** − 1, 1 **bara**	30 **Pa** − 110 **kPa**[d]	4.4 **psi** − 16.0 **psi**
Pressure Accuracy	±0.25 %			±0.25 kPa			±0.4 kPa			±0.15 kPa		
Output Resolution	12 bits			8 - 12 bits			20 bits			8 - 12 bits		
Long term stability	N/A			$\pm 0.1 \frac{\text{kPa}}{\text{yr}}$			$\pm 0.1 \frac{\text{kPa}}{\text{yr}}$			$\pm 0.1 \frac{\text{kPa}}{\text{yr}}$		
Max Pressure[d]	N/A			6 bar	600 kPa	87.0 psi	5 bar	500 kPa	72.5 kPa	6 bar	600 kPa	87.0 psi
Noise	N/A			1.2 (4096), 1.8 (2048), 2.7 (1024), 4.2 (512), 6.5 (256)		Pa-rms (OSR)	19 (1) 1.5 (128)		Pa-rms (OSR)	2.4 (4096), 3.6 (2048), 5.4 (1024), 8.4 (512), 13.0 (256)		Pa-rms (OSR)
Operating Voltage V_s/V_{DD}	**Min**	**Typical**	**Max**	**Min**	**Typical**	**Max**	**Min**	**Typical**	**Max**	**Min**	**Typical**	**Max**
	3.0 V	3,3 V	3.6 V	1.8 V	3.0 V	3.6 V	1.95 V	2.5 V	3.6 V	1.8 V	3.0 V	3.6 V
Interface Voltage V_{DDIO}	N/A			N/A			1.62 V	1.8 V	3.6 V	N/A		
Output Data Rate (ODR)	**SPI** (Max)	**I2C** (Max)		**SPI** (Max)	**I2C** (Max)		**I2C** $f_{SCL}1$ (with $C_{Bus} = 20$ pF)	**I2C** $f_{SCL}2$ (with $C_{Bus} = 400$ pF)		**SPI** (Max)	**I2C** (Max)	
	800 kHz	400 kHz		20 MHz	400 kHz		400 kHz	4 MHz		20 MHz	400 kHz	
Current Consumption	**Supply** (typical)	**Supply** (max)		**Supply**	**Peak**	**Standby**	**Operational**	**Peak**	**Standby**	**Supply**	**Peak**	**Standby**
	3.1 mA	3.9 mA		12.5 µA (4096)[e]	1.4 mA	0.14 µA	265 µA[e]	2 mA	2 µA	12.5 µA[e]	1.4 mA	0.14 µA
Power Consumption	**For** $V_{DD} = 3.3$ **V**			**For** $V_{DD} = 3.3$ **V**			**For** $V_{DD} = 3.3$ **V**			**For** $V_{DD} = 3.3$ **V**		
	10.23 mW	12.87 mW		42.24 µW	4.62 mW	0.462 µW	844.8 µW	6.6 mW	6.6 µW	41.25 µW	4.62 mW	0.462 µW
Temperature Range	-20 ... +85 °C			-40 ... +85 °C			-40 ... +85 °C			-40 ... +85 °C		
Temperaturesensor Specs												
Measurement Range				-40 ... +85 °C			-40 ... +85 °C			-40 ... +85 °C		
Accuracy				**Min**	**-20 ... +85 °C**	**Max**	Max Offset			**Min**	**-20 ... +85 °C**	**Max**
				-0.8 °C	±2.0 °C	+0.8 °C	±3 °C			-0.8 °C	±2.0 °C	+0.8 °C
Resolution				24 bits			12 bits			24 bits		
Noise				0.002 (4096), 0.003 (2048), 0.005 (1024), 0.008 (512), 0.012 (256)		°C (OSR)	N/A			0.002 (4096), 0.003 (2048), 0.005 (1024), 0.008 (512), 0.012 (256)		°C (OSR)

[a]psi values are rounded to a precision of one tenth

[a]Features an altimeter mode

[b]Extended usable range of 1 kPa - 120 kPa. The ADC shows linear behaviour in the above mentioned range, but can be extended to this range with custom calibrations.

[c]Extended usable range of 20 kPa - 120 kPa with similar ADC behaviour to the MS561101BA03-50

[d]Extended usable range of 1 kPa - 200 kPa with similar ADC behaviour to the MS561101BA03-50

[d]Referring to the most pressure it can sustain without breaking

[e]Maximum operational supply. The Consumption is dependent on OSR.